Unit 1

Introduction to Artificial Intelligence

Definitions of AI, Applications of Artificial Intelligence, Concept of Modeling, Inference and Learning. Introduction to Machine learning and Deep learning as a subset of AI. Intelligent agents, concept of rationality, structure of agents, Environment, Properties of task environment. Real world Examples of agents and environments

1. What is AI?
2. What is Acting humanly?
3. What is Thinking Humanly?
4. What is Thinking Rationally?
5. What is Acting Rationally?

Unit 1

1. What is AI?

Definitions of artificial intelligence according to eight textbooks are shown in Figure 11.1. These definitions vary along two main dimensions. Roughly, the ones on top are concerned with thought processes and reasoning, whereas the ones on the bottom address behaviour. The definitions on the left measure success in terms of fidelity to human performance, whereas the ones on the right measure against an ideal concept of intelligence, which we will call rationality. A system is rational if it does the "right thing," given what it knows.

Thinking humanly	Thinking rationally
Acting humanly	Acting rationally

Systems that think like humans:

This concept refers to AI systems that aim to replicate human thought processes and cognitive functions. The goal is to develop systems that can understand, reason, learn, and make decisions in a manner similar to how humans do. These systems might employ techniques inspired by human cognition, such as pattern recognition, inference, and learning from experience.

Systems that think rationally:

This idea is associated with the pursuit of logical and rational decision-making in AI systems. Instead of mimicking human thought processes, these systems aim to adhere to principles of formal logic and reason. The focus is on creating systems that can derive conclusions based on logical inference, ensuring that their decisions follow a rational and coherent pattern.

Systems that act like humans:

This concept pertains to AI systems that aim to emulate human behavior in terms of actions and interactions with the environment. The goal is to design systems that can perform tasks and exhibit behavior in a manner similar to how humans would. This involves understanding and replicating not just cognitive processes but also the physical actions and responses that humans exhibit.

Systems that act rationally:
This approach involves designing AI systems that make decisions and take actions based on rational principles. The focus is on achieving optimal outcomes or goals, without necessarily replicating human thought processes. These systems aim for logical decision-making that leads to the best possible results.

A human-centred approach must be an empirical science, involving hypothesis and experimental confirmation. A rationalist approach involves a combination of mathematics and engineering. Each group has both disparaged and helped the other

2. What is Acting humanly?

The Turing Test, proposed by Alan Turing (195O), was designed to provide a satisfactory operational definition of intelligence. Rather than proposing a long and perhaps controversial list of qualifications required for intelligence, he suggested a test based on indistinguishability from undeniably intelligent entities-human beings

The computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or not. That programming a computer to pass the test provides plenty to work on. The computer would need to possess the following capabilities:

Natural language processing to enable it to communicate successfully in English.
knowledge representation to store what it knows or liars;
Automated reasoning to use the stored information to answer questions and to draw new conclusions;
machine learning to adapt to new circumstances and to detect and extrapolate patterns

Turing's test deliberately avoided direct physical interaction between the interrogator and the computer, because physical simulation of a person is unnecessary for intelligence. However, the so-called total Turing Test includes a video signal so that the interrogator can test the subject's perceptual abilities, as well as the opportunity for the interrogator to pass physical objects "through the hatch." To pass the total Turing Test, th~ec omputer will need

computer vision to perceive objects, and
robotics to manipulate objects and move about.

3. What is Thinking Humanly?

If we are going to say that a given program thinks like a human, we must have some way of determining how humans think. We need to get inside the actual workings of human minds.There are two ways to do this: through introspection~-trying to catch our own thoughts as they go by-and through psychological experiments.

Once we have a sufficiently precise theory of the mind, it becomes possible to express the theory as a computer program. If the program's input/output and timing behaviors match corresponding human behaviors, that is evidence that some of the program's mechanisms could also be operating in humans

For example, Allen Newel1 and Herbert Simon, who developed GPS, the "General Problem Solver" (Newel1 and Simon, 1961), were not content to have their program solve problems correctly.They were more concerned with comparing the trace of its reasoning steps to traces of human subjects solving the same problems.

The interdisciplinary field of cognitive science brings together computer models from AI and experimental techniques from psychology to try to construct precise and testable theories of the workings of the human mind

4. What is Thinking Rationally?

The Greek philosopher Aristotle was one of the first to attempt to codify "right thinking," that SYLLOGISMS is, irrefutable reasoning processes. His syllogisms provided patterns for argument structures that always yielded correct conclusions when given correct premises-for example, "Socrates is a man; all men are mortal; therefore, Socrates is mortal." These laws of thought were LOGIC supposed to govern the operation of the mind; their study initiated the field called logic.

By 1965, programs existed that could, in principle, solve any solvable problem described in logical notation

There are two main obstacles to this approach. First, it is not easy to take informal knowledge and state it in the formal terms required by logical notation, particularly when the knowledge is less than 100% certain.

Second, there is a big difference between being able to solve a problem "in principle" and doing so in practice. Even problems with just a few dozen facts can exhaust the computational resources of any computer unless it has some guidance as to which reasoning steps to try first.

5. What is Acting Rationally?

An agent is just something that acts . But computer agents are expected to have other attributes that distinguish them from mere "programs," such as operating under autonomous control or perceiving their environment.

A rational agent is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome.

In the "laws of thought" approach to AI, the emphasis was on correct inferences. Making correct inferences is sometimes part of being a rational agent, because one way to act rationally is to reason logically to the conclusion that a given action will achieve one's goals and then to act on that conclusion. On the other hand, correct inference is not all of rationality, because there are often situations where there is no provably correct thing to do, yet something must still be done. There are also ways of acting rationally that cannot be said to involve inference. For example, recoiling from a hot stove is a reflex action that is usually more successful than a slower action taken after careful deliberation.

For these reasons, the study of A1 as rational-agent design has at least two advantages.

First, it is more general than the "laws of thought" approach, because correct inference is just one of several possible mechanisms for achieving ratiornality. Second, it is more amenable to scientific development than are approaches based on human behavior or human thought because the standard of rationality is clearly defined and conipletely general.

6. Application of AI?

Autonomous planning and scheduling

A hundred million miles from Earth, NASA's Remote Agent program became the first on-board autonomous planning program to control the scheduling of operations for a spacecraft (Jonsson et al., 2000). Remote Agent generated plans from high-level goals specified from the ground, and it monitored the operation of the spacecraft as the plans were executed-detecting, diagnosing, and recovering from problems as they occurred.

Game playing

IBM's Deep Blue became the first computer program to defeat the world champion in a chess match when it bested Garry Kasparov by a score of 3.5 to 2.5 in an exhibition match (Goodman and Keene, 1997). Kasparov said that he felt a "new kind of intelligence" across the board from him. Newsweek magazine described the match as "The brain's last stand." The value of IBM's stock increased by $18 billion.

Autonomous control

The ALVINN computer vision system was trained to steer a car to keep it following a lane. St was placed in CMU's NAVLAB computer-controlled minivan and used to navigate across the United States-for 2850 miles it was in control of steering the vehicle 98% of the time. A human took over the other 2%, mostly at exit ramps. NAVLABh as video cameras that transmit road images to ALVINN, which then computes the best direction to steer, based on experience from previous training runs.

Robotics

Many surgeons now use robot assistants in microsurgery. HipNav (DiGioia et al., 1996) is a system that uses computer vision techniques to create a three-dimensional model of a patient's internal anatomy and then uses robotic control to guide the insertion of a hip replacement prosthesis.

Logistics Planning

During the Persian Gulf crisis of 1991, U.S. forces deployed a

Dynamic Analysis and Replanning Tool, DART (Cross and Walker, 1994), to do automated logistics planning and scheduling for transportation. This involved up to 50,000 vehicles, cargo, and people at a time, and had to account for starting points, destinations, routes, and conflict resolution among all parameters. The AI planning techniques allowed a plan to be generated in hours that would have taken weeks with older methods. The Defense Advanced Research Project Agency (DARPA) stated that this single application more than paid back DARPA's 30-year investment in AI.

7. What is Agent?

An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

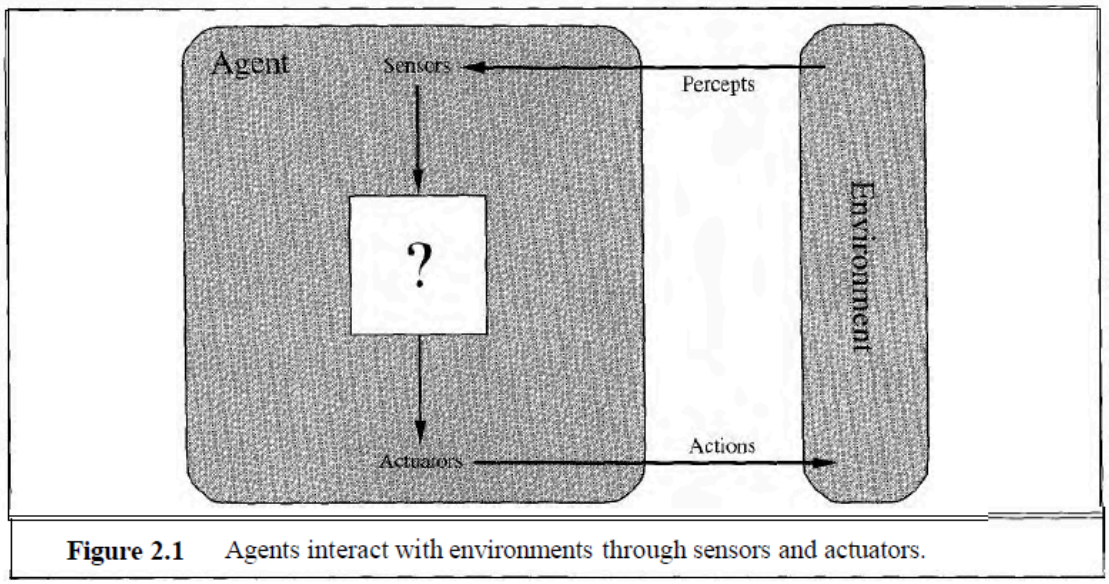


Figure 2.1 Agents interact with environments through sensors and actuators.

A human agent has eyes, ears, and other organs for sensors and hands, legs, mouth, and other body parts for actuators. A robotic agent might have cameras and infrared range finders for sensors and various motors for actuators. A software agent receives keystrokes, file contents, and network packets as sensory inputs and acts on the environment by displaying on the screen, writing files, and sending network packets.

We use the term percept to refer to the agent's perceptual inputs at any given instant. An agent's percept sequence is the complete history of everything the agent has ever perceived.
In general, an agent's choice of action at any given instant can depend on the entire percept sequence observed to date. If we can specify the agent's

choice of action for every possible percept sequence, then we have said more or less everything there is to say about the agent.Mathematically speaking, we say that an agent's behaviour is described by the **agent function** that maps any given percept sequence to an action.

Given an agent to experiment with, we can, in principle, construct this table by trying out all possible percept sequences and recording which actions the agent does in response.' The table is, of course, an external characterization of the agent. Internally, Ihe agent function for an artificial agent will be implemented by an **agent program**. It is important to keep these two ideas distinct. The agent function is an abstract mathematical description; the agent program is a concrete implementation, running on the agent architecture.

To illustrate these ideas, we will use a very simple example-the vacuum-cleaner world shown in Figure 2.2. This world is so simple that we can describe everything that happens; it's also a made-up world, so we can invent many variations. This particular world has just two locations: squares A and B. The vacuum agent perceives which square it is in and1 whether there is dirt in the square. It can choose to move left, move right, suck up the dirt, or do nothing. One very simple agent function is the following: if the current square is dirty, then suck, otherwise move to the other square. A partial tabulation of this agent function is shown in Figure 2.3.

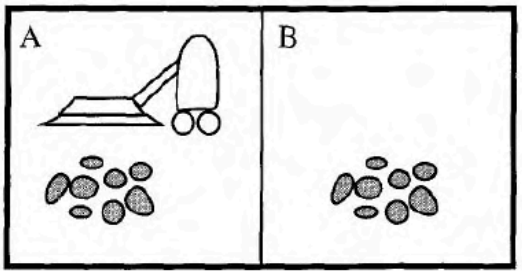


Figure 2.2 A vacuum-cleaner world with just two locations.

Percept sequence	Action
[A, *Clean*]	*Right*
[A, *Dirty*]	Suck
[*B*,*Clean*]	*Left*
[*B*,*Dirty*]	Suck
[*A*,*Clean*],[*A*,*Clean*]	*Right*
[*A*,*Clean*],[*A*,*Dirty*]	Suck
[*A*,*Clean*],[*A*,*Clean*],[*A*,*Clean*]	*Right*
[*A*,*Clean*],[*A*,*Clean*],[*A*,*Dirty*]	Suck

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

8. What is a rational Agent?

A rational agent is one that does the right thing-conceptually speaking, every entry in the table for the agent function is filled out correctly. Obviously, doing the right thing is better than doing the wrong thing, but what does it mean to do the right thing? As a first approximation, we will say that the right action is the one that will cause the agent to be most successful. Therefore, we will need some way to measure success

Performance Measure

The performance measure is a criterion or standard used to evaluate the success of an agent's behaviour. When an agent is plunked down in an environment, it generates a sequence of actions according to the percepts it receives. This sequence of actions causes the environment to go through a sequence of states. If the sequence is desirable, then the agent has performed well.

Consider the simple vacuum-cleaner agent that cleans a square if it is dirty and moves to the other square if not; this is the agent function tabulated in Figure 2.3. Is this a rational agent? That depends! First, we need to say what the performance measure is, what is known about the environment, and what sensors and actuators the agent has. Let us assume the following:

> The performance measure awards one point for each clean square at each time step, over a "lifetime" of 1000 time steps.
>
> The "geography" of the environment is known a priori (Figure 2.2) but the dirt distribution and the initial location of the agent are not. Clean squares stay clean and sucking cleans the current square. The Lefi and Right actions move the agent left and right except when this would take the agent outside the environment, in which case the agent remains where it is.
>
> The only available actions are Left, Right, Suck, and NoOp (do nothing). The agent correctly perceives its location and whether that location contains dirt.
>
> We claim that under these circumstances the agent is indeed rational; its expected performance is at least as high as any other agent's. Exercise 2.4 asks you to prove this.

One can see easily that the same agent would be irrational under different circumstances. For example, once all the dirt is cleaned up it will oscillate needlessly back and forth; if the performance measure includes a penalty of one point for each movement left or right, the agent will fare poorly. A better agent for this case would do nothing once it is sure that all the squares are clean. If clean squares can become dirty again, the agent should occasionally check and re-clean them if needed. If the geography of the environment is unknown, the agent will need to explore it rather than stick to squares A and B.

Omniscience

We need to be careful to distinguish between rationality and omniscience. An omniscient agent knows the actual outcome of its actions and can act accordingly; but omniscience is impossible in reality. Consider the following example: I am walking along the Champs ElysCes one day and I see an old friend across the street. There is no traffic nearby and I'm not otherwise engaged, so, being rational, I start to cross the street. Meanwhile, at 33,000

feet, a cargo door falls off a passing air~inera,~nd before I make it to the other side of the street I am flattened. Was I irrational to cross the street? It is unlikely that my obituary would read "Idiot attempts to cross street."

This example shows that rationality is not the same as perfection. Rationality maximizes expected performance, while perfection maximizes actual performance

Information Gathering

The rational choice depends only on the percept sequence to date. We must also ensure that we haven't inadvertently allowed the agent to engage in decidedly underintelligent activities.]For example, if an agent does not look both ways before crossing a busy road, then its percept sequence will not tell it that there is a large truck approaching at high speed. Does our definition of rationality say that it's now OK to cross the road? Far from it! First, it would not be rational to cross the road given this uninformative percept sequence: the risk of accident from crossing without looking is too great. Second, a rational agent should choose the "looking" action before stepping into the street, because looking helps maximize the expected performance. Doing actions in order ro modify future percepts-sometimes called information gathering-is an important part of rationality

Autonomy

A rational agent should be autonomous-it should learn what it can to compensate for partial or incorrect prior knowledge.
For example, a vacuum-cleaning agent that learns to foresee where and when additional dirt will appear will do better than one that does not. when the agent has had little or no experience, it would have to act randomly unless the designer gave some assistance. So, just as evolution provides animals with enough built-in reflexes so that they can survive long enough to lear for themselves, it would be reasonable to provide an artificial intelligent agent with some initial knowledge as well as an ability to learn.

After sufficient experience of its environment, the behavior of a rational agent can become effectively independent of its prior knowledge.
Hence, the incorporation of learning allows one to design a single rational agent that will succeed in a vast variety of environments.

9. What is a task Environment and a PEAS EXAMPLE ?

The task environments, which are essentially the "problems" to which rational agents are the "solutions.Simple vacuum-cleaner agent, we had to specify the performance measure, the environment, and the agent's actuators and sensors. We will group all these together under the heading of the task environment. For the acronymically minded, we call this the PEAS (Performance, Environment, Actuators, Sensors) description.

In designing an agent, the first step must always be to specify the task environment as fully as possible. The vacuum world was a simple example; let us consider a more complex problem: an automated taxi driver.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe: fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

SOME SOFTWARE AGENTS

For example, a robot designed to inspect parts as they come by on a conveyor belt can make use of a number of simplifying assumptions: that the lighting is always just so, that the only thing on the conveyer belt will be parts of a kind that it knows about, and that there are only two actions (accept or reject).

In contrast, some software agents (or software robots or softbots) exist in rich, un-limited domains. Imagine a softbot designed to fly a flight simulator for a large commercial airplane. The simulator is a very detailed, complex environment including other aircraft and ground operations, and the software agent must choose from a wide variety of actions in real time.

10. The Various Environment Types?

a. Fully Observable

If an agent's sensors give it access to the complete state of the environment at each point in time, then we say that the task environment is fully ob~ervable.A task environment is effectively fully observable if the sensors detect all aspects that are relevant to the choice of action; relevance, in turn, depends on the performance measure. Fully observable environments are convenient because the agent need not maintain any internal state to keep track of the world. An environment might be partially observable because of noisy and inaccurate sensors or because parts of the state are simplly missing from the sensor data-for example, a vacuum agent with only a local dirt sensor cannot tell whether there is dirt in other squares, and an automated taxi cannot see what other drivers are thinking.

Example: Chess

- Agent: Chess player
- Environment: Chess board
- Observations: The agent can see the positions of all pieces on the board at any given time.
- Fully Observable: The entire state of the chess board is visible to both players.

Example: Poker

- Agent: Poker player
- Environment: Poker table
- Observations: The agent can only see its own cards and the community cards on the table.
- Partially Observable: The player lacks information about the cards held by opponents.

b. Deteerministic vs stochastic

If the next state of the environment is completely determined by the current state and the action executed by the agent, then we say the environment is deterministic; otherwise, it is stochastic. In principle, an agent need not worry about uncertainty in a fully observable, deterministic environment. If the

environment is partially observable, however, then it could appear to be stochastic.

Example: Deterministic Pendulum

- System: Simple pendulum
- Behavior: The motion of a simple pendulum is determined entirely by its initial conditions (initial angle and initial velocity) and the laws of physics.
- Predictability: If you know the initial conditions and the physical parameters of the pendulum, you can precisely predict its position and velocity at any point in the future using deterministic equations.

Example: Stochastic Process - Coin Toss

- System: Tossing a fair coin
- Behavior: The outcome of a single coin toss is influenced by random factors such as air resistance, initial orientation, and surface characteristics.
- Uncertainty: While there are only two possible outcomes (heads or tails), the specific result of any given toss is uncertain and follows a probability distribution.
- Probability: In a fair coin toss, the probability of getting heads or tails is equal, but the actual outcome is not determined until the coin lands.

c. Episodic vs. sequential

In an episodic task environment, the agent's experience is divided into atomic episodes. Each episode consists of the agent perceiving and then performing a single action. Crucially, the next episode does not depend on the actions taken in previous episodes. In episodic environments, the choice of action in each episode depends only on the episode itself. Many classification tasks are episodic. For example, an agent that has to spot defective parts on an assembly line bases each decision on the current part, regardless of previous decisions; moreover, the current decision doesn't affect whether the nextpart is defective. In sequential environments, on the other hand, the current decision could affect all future decisions. Chess and taxi driving are sequential: in both cases, short-term actions can have long-term consequences. Episodic environments are much simpler than sequential environments because the agent does not need to think ahead.

d. Static vs, dynamic

If the environment can change while an agent is deliberating, then we say the environment is dynamic for that agent; otherwise, it is static. Static environments are easy to deal with because the agent need not keep looking at the world while it is deciding on an action, nor need it worry about the passage of time. Dynamic environments, on the other hand, are continuously asking the agent what it wants to do; if it hasn't decided yet, that counts as deciding to do nothing.

If the environment itself does not change with the passage of time but the agent's performance score does, then we say the environment is semidynamic. Taxi driving is clearly dynamic: the other cars and the taxi itself keep moving while the driving algorithm dithers about what to do next. Chess, when played with a clock, is semidynamic. Crossword puzzles are static.

e. Discrete vs. continuous.

A discrete-state environment such as a chess game has a finite number of distinct states. Chess also has a discrete set of percepts and actions. Taxi driving is a continuous state and continuous-time problem: the speed and location of the taxi and of the other vehicles sweep through a range of continuous values and do so smoothly over time. Taxi-driving actions are also continuous (steering angles, etc.).

f. Single agent vs. multiagent

An agent solving a crossword puzzle by itself is clearly in a single-agent environment, whereas an agent playing chess is in a two-agent environment. There are, however, some subtle issues. First, we have described how an entity may be viewed as an agent, but we have not explained which entities must be viewed as agents. Does an agent A (the taxi driver for example) have to treat an object B (another vehicle) as an agent, or can it be treated merely as a stochastically behaving object, analogous to waves at the beach or leaves blowing in the wind?

The key distinction is whether B's behavior is best described as maximizing a performance measure whose value depends on agent A's behavior. For example, in chess, the opponent entity B is trying to maximize its performance measure, which, by the rules of chess, minimizes agent A's performance measure. Thus, chess is a **competitive multiagent environment.** In the taxi-driving environment, on the other hand, avoiding collisions maximizes the performance measure of all agents, so it is a **partially cooperative multiagent environment**.

11. Structure of Agent and Agent Program Example ?

The job of A1 is to design the agent program that implements the

agent function mapping percepts to actions. We assume this program will run on some sort of computing device with physical sensors and actuators we call this the architecture:

agent = architecture +program .

Obviously, the program we choose has to be one that is appropriate for the architecture. If the program is going to recommend actions like Walk, the architecture had better have legs. The architecture might be just an ordinary PC, or it might be a robotic car with several onboard computers, cameras, and other sensors.

Agent Program

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  static: percepts, a sequence, initially empty
          table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It keeps track of the percept sequence using its own private data structure.

For example, Figure 2.7 shows a rather trivial agent program that keeps track of the percept sequence and then uses it to index into a table of actions to decide what to do. The table represents explicitly the agent function that the agent program embodies. To build a rational agent in this way, we as designers must construct a table that contains the appropriate action for every possible percept sequence.

The key challenge for A1 is to find out how to write programs that, to the extent possible, produce rational behavior from a small amount of code rather than from a large number of table entries. We have many examples showing that this can be done successfully in other areas: for example, the huge tables of square roots used by engineers and schoolchildren prior to the 1970s have now been replaced by a five-line pro~gram for Newton's method running on electronic calculators.

Program for a Vaccum -cleaner Agent

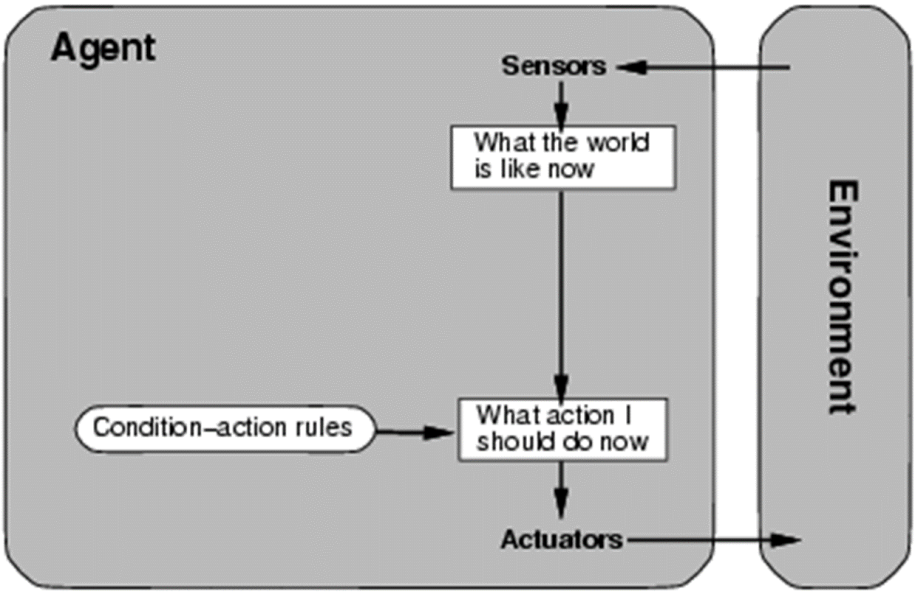
```
function REFLEX-VACUUM-AGENT([location,status]) returns an action

  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

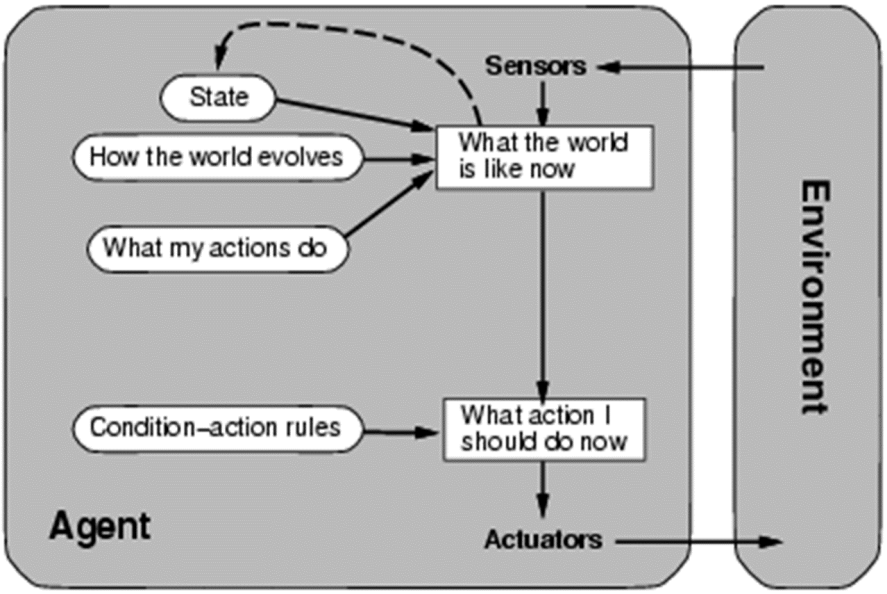
12. What are the various types of Agents?

Theory: Gate Smashers

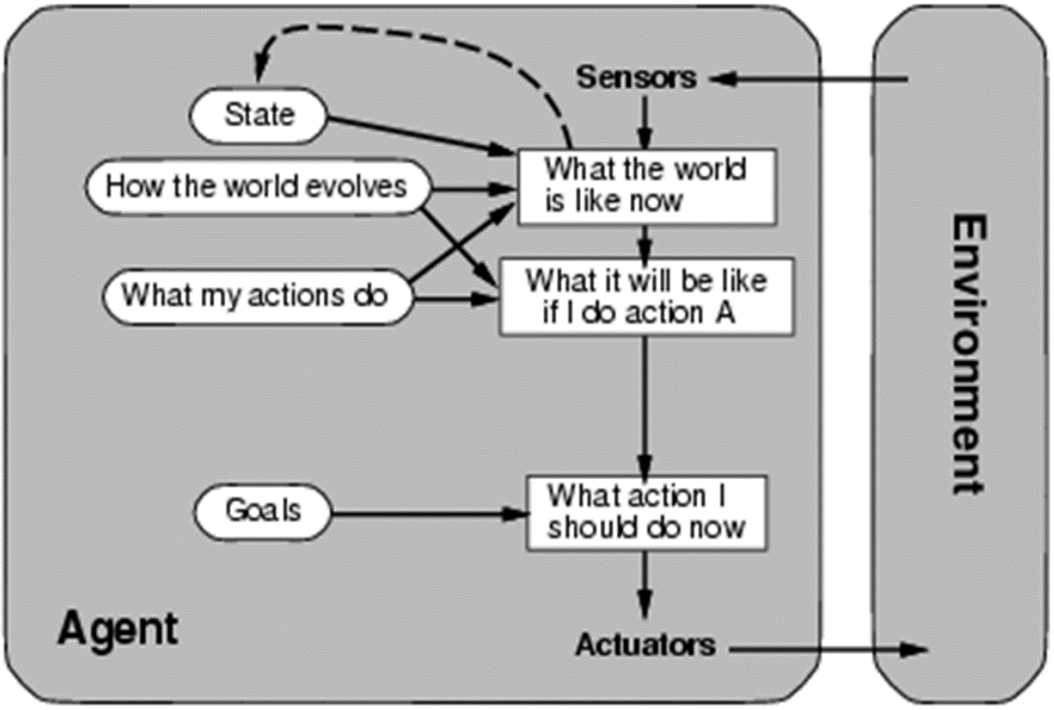
a. Simple reflex agents



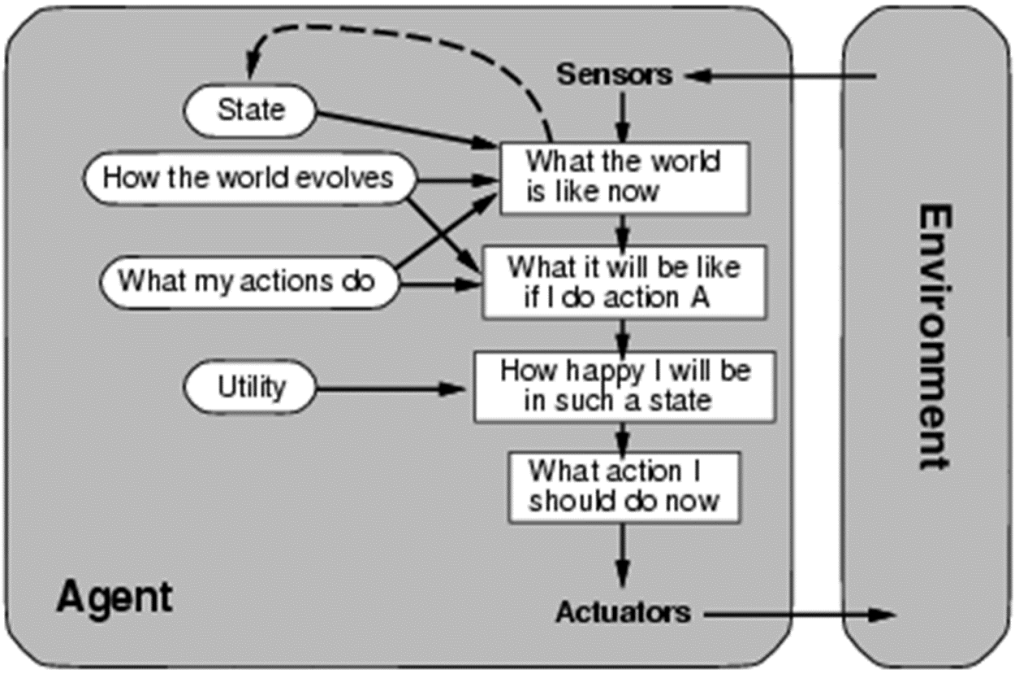
b. Model-based reflex agents'



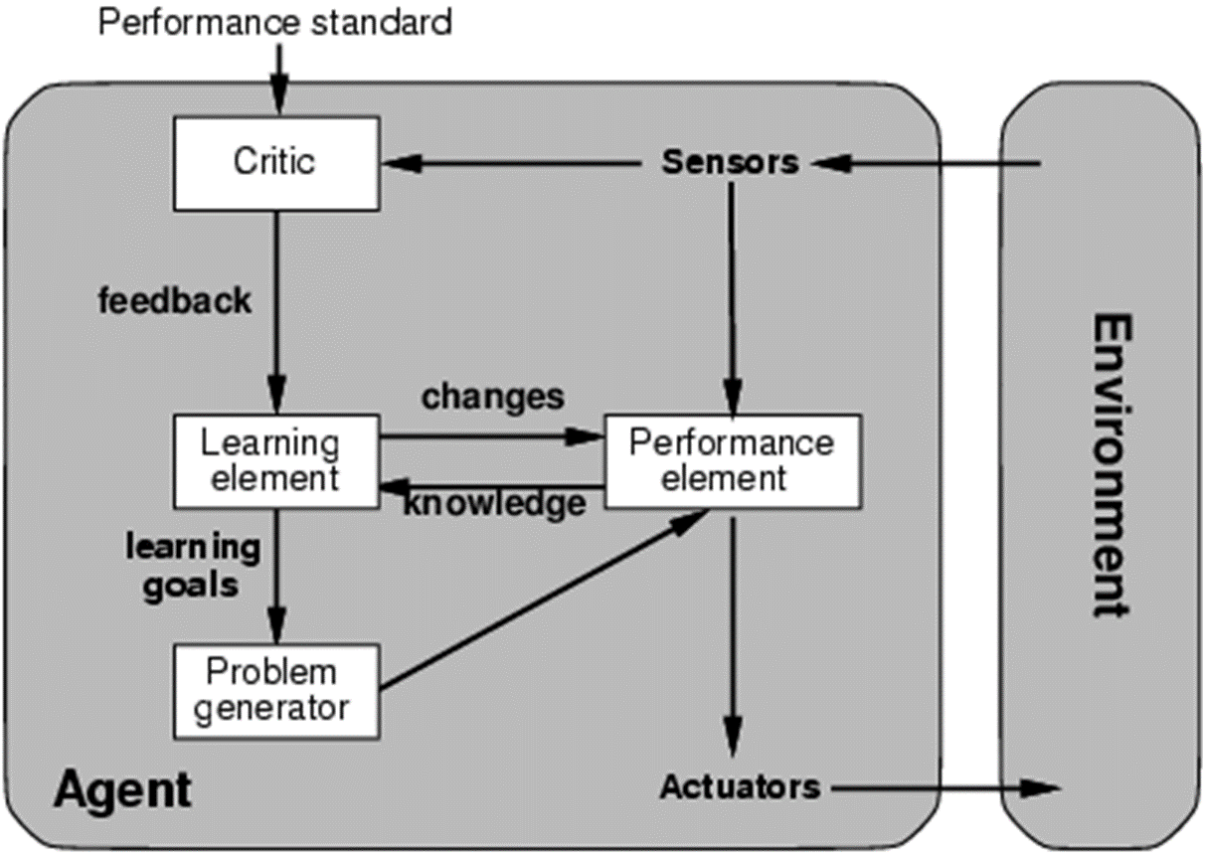
c. Goal-based agents



d. Utility-based agents



13. What are Learning agents?



Unit 2

Unit 2

1. What is Problem Solving Agents/ Problem Formulation?
2. Searching for Solution?
3. Uniformed Search : BFS / DFS?
4. What is Heuristic in AI?
5. What is Greedy Breadth First Search?
6. What is A* ?
7. Proof for A* is admissible?

8. Local Search algorithm

https://youtu.be/DwWlAiZ2PP0?si=ywrGzhrNFz7b7iz6

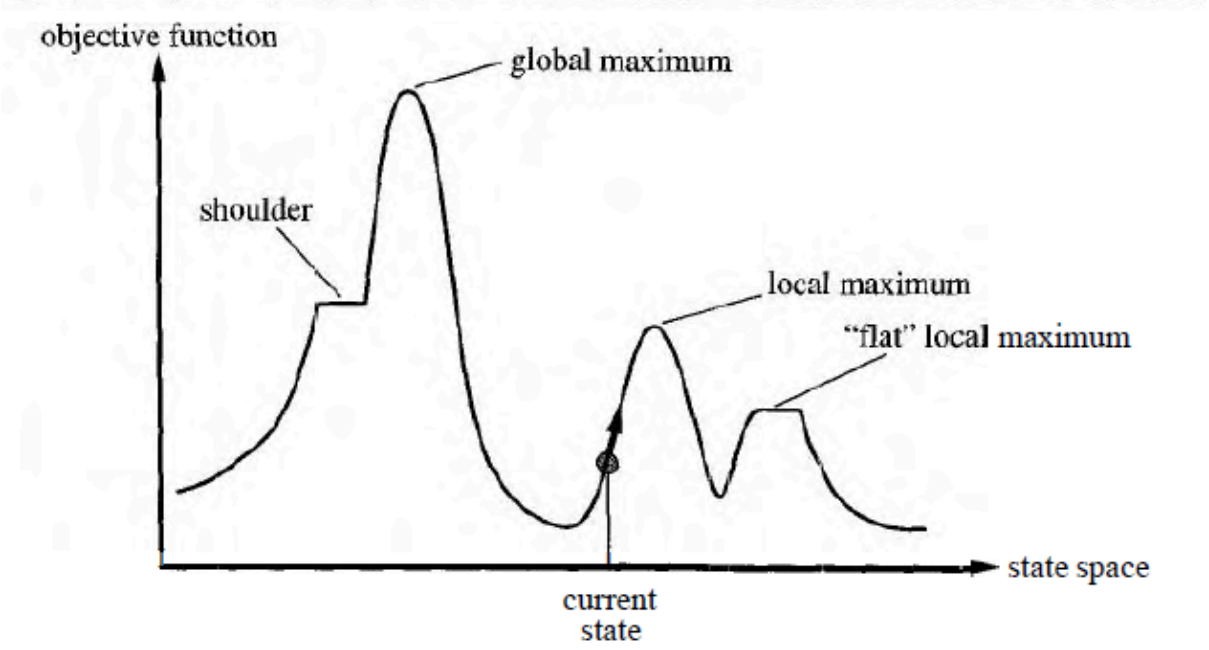
The search algorithms that we have seen so far are designed to explore search spaces systematically. This systematicity is achieved by keeping one or more paths in memory and by recording which alternatives have been explored at each point along the path and which have not.

When a goal is found, the path to that goal also constitutes a solution to the problem. In many problems, however, the path to the goal is irrelevant. For example, in the 8-queens problem (see page 66), what matters is the final configuration of queens, not the order in which they are added. This class of problems includes many important applications such as integrated-circuit design, factory-floor layout, job-shop scheduling, automatic programming,telecommunications network optimization, vehicle routing, and portfolio management.

If the path to the goal does not matter, we might consider a different class of algorithms, ones that do not worry about paths at all. Local search algorithms operate using a single current state (rather than multiple paths) and generally move only to neighbors of that state. Typically, the paths followed by the search are not retained. Although local search algorithms are not systematic, they have two key advantages: (1) they use very little memory-usually a constant amount; and (2) they can often find reasonable solutions in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable.

In addition to finding goals, local search algorithms are useful for solving pure optimization problems, in which the aim is to find the best state according to an objective function.

If elevation corresponds to cost, then the aim is to find the lowest valley a global minimum if elevation corresponds to an objective function, then the aim is to find the highest peak-a global maximum,. (You can convert from one to the other just by inserting a minus sign.) Local search algorithms explore this landscape.

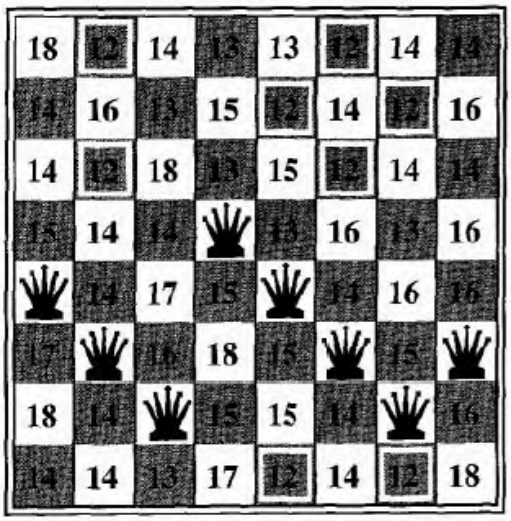


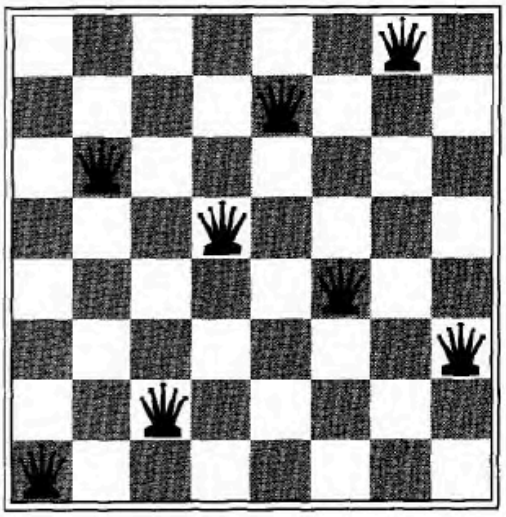
1. Hill Climb Search

https://youtu.be/3SiWtAnUROs?si=NLGa7rxXjuiThGNE

Hill-Climbing Search

- **Step 1:** Evaluate the initial state, if it is goal state then return success and Stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state.
- **Step 4:** Check new state:
  - If it is goal state, then return success and quit.
  - Else if it is better than the current state then assign new state as a current state.
  - Else if not better than the current state, then return to step2.
- **Step 5:** Exit.





The heuristic cost function h is the number of pairs of queens that are attacking each other, either directly or indirectly. The global minimum of this function is zero, which occurs only at perfect solutions. Figure 4.12(a) shows a state with h = 17.
The figure also shows the values of all its successors, with the best successors having h = 12. Hill-climbing algorithms typically choose randomly among the set of best successors, if there is more than one.

Hill climbing is sometimes called greedy local s'earch because it grabs a good neighbor state without thinking ahead about where to go next. Althoughg Greed is considered one of the seven deadly sins, it turns out that greedy algorithms often perform quite well. Hill climbing often makes very rapid progress towards a solution, be'cause it is usually quite easy to improve a bad state. For example, from the state in Figure 4.12(a), it takes just five steps to reach the state in Figure 4.12(b), which has h = 1 and is very nearly a solution.

Unfortunately, hill climbing often gets stuck for the following reasons:

Local maxima: a local maximum is a peak that is higher than each of its neighboring states, but lower than the global maximum. H[ill-climbing algorithms that reach the vicinity of a local maximum will be drawn upwards towards the peak, but will then be stuck with nowhere else to go. Figure 4.10 illusfrates the problem schematically. More concretely, the state in Figure

4.12(b) is in fact a local maximum (i.e., a local minimum for the cost h); every move of a single queen makes the situation worse.

Ridges: a ridge is shown in Figure 4.13. Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.

Plateaux: a plateau is an area of the state space landscape where the evaluation function is flat. It can be a flat local maximum, from which no uphill exit exists, or a shoulder, from which it is possible to make progress. (See Figure 4.10.) A hill-climbing search might be unable to find its way off the plateau.

Sideway Moves

The algorithm in Figure 4.11 halts if it reaches a plateau where the best successor has the same value as the current state. Might it not be a good idea to keep going-to allow a sideways move in the hope that the plateau is really a shoulder, as shown in Figure 4. lo? The answer is usually yes, but we must take care. If we always allow sideways moves when there are no uphill moves, an infinite loop will occur whenlever the algorithm reaches a flat local maximum that is not a shoulder.

One common solution is to put a limit on the number of consecutive sidleways moves allowed. For example, we could allow up to, say, 100 consecutive sideways moves in the %queens problem. This raises the percentage of problem instances solved by hill-climbing from 14% to 94%. Success comes at a cost: the algorithm averages
roughly 21 steps for each successful instance and 64 for each failure.

Variant Of HC

Basic Hill Climbing: This is the simplest form of hill climbing where the algorithm iteratively moves to the neighboring solution that maximally improves the current solution. It terminates when no better solution is found in the neighborhood.

Steepest Ascent Hill Climbing: In this variant, the algorithm examines all neighboring solutions and selects the one that maximizes the objective function the most. It continues this process until a local maximum is reached.

Random Restart Hill Climbing: This variant aims to overcome the problem of getting stuck in local maxima by restarting the hill climbing process from multiple random starting points. It increases the likelihood of finding the global maximum.

Simulated Annealing: Simulated annealing is a probabilistic variant of hill climbing that allows the algorithm to accept worse solutions with a certain probability. Initially, it accepts worse solutions with a high probability and gradually decreases this probability as the algorithm progresses. This helps to escape local maxima and explore the solution space more effectively.

Stochastic Hill Climbing: Unlike basic hill climbing, stochastic hill climbing selects a random neighboring solution to move to, even if it is worse than the current solution. This randomness can help in exploring the solution space more broadly.

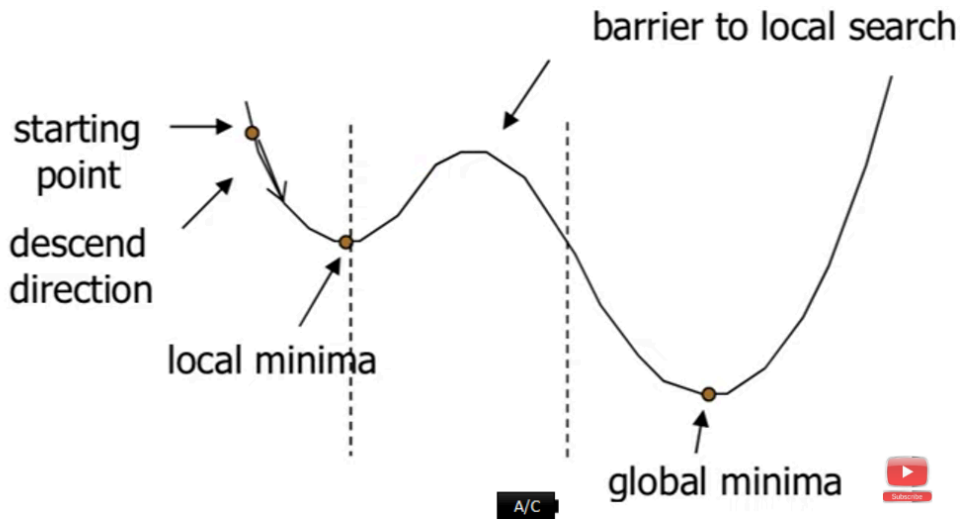
2. Simulated Annealing Search

https://youtu.be/uK6nbn7hArQ?si=xRCOVjDWKjIHwcsb

https://youtu.be/21EDdFVMz8I?si=8pMi1-ECryAQvqnS

A hill-climbing algorithm that never makes "downhill" moves towards states with lower value (or higher cost) is guaranteed to be incomplete, because it can get stuck on a local maximum.
In contrast, a purely random walk-that is, moving to a successor chosen uniformly at random from the set of successors-is complete, but extremely inefficient.
Therefore, it seems reasonable to try to combine hill climbing with a random walk in some way that yields both efficiency and completeness.

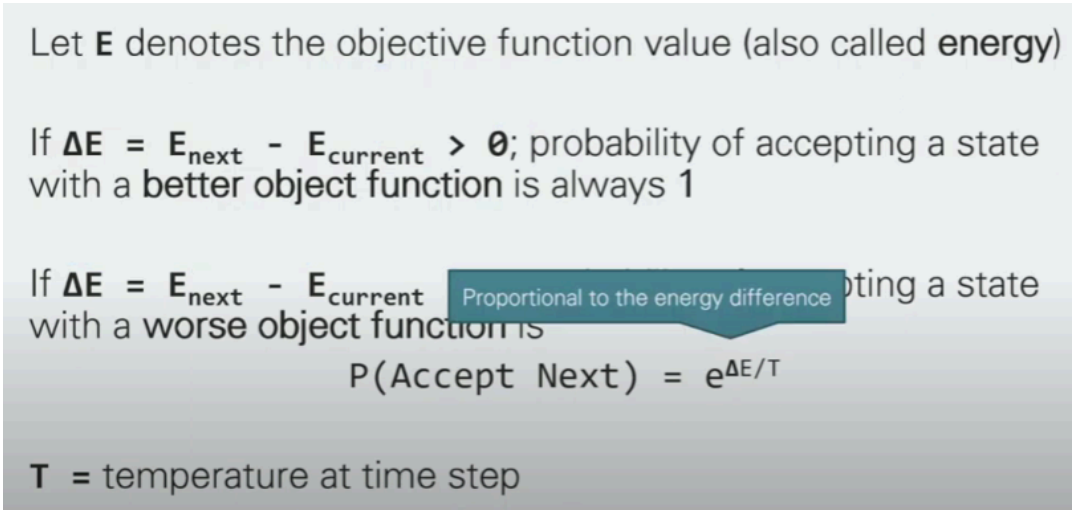
Simulated annealing is such an algorithm.
In metallurgy, annealing is the process used to temper or harden metals and glass by heating them to a high temperature and then gradually cooling them, thus alllowing the material to coalesce into a low-energy crystalline state.



imagine the task of getting a ping-pong ball into the deepest crevice in a bumpy surface. If we just let the ball roll, it will come to rest at a local minimum. If we shake the surface, we can bounce the ball out of the local minimum. The trick is to shake just hard enough to bounce the ball out of local minima, but not hard enough to dislodge it from the global minimum.
The simulated annealing solution is to start by shaking hard (i.e., at a high temperature) and then gradually reduce the intensity of the shaking (i.e., lower the temperature).

ALGO

The innermost loop of the simulated-annealing algorithm (Figure 4.14) is quite similar to hill climbing. Instead of picking the best move, however, it picks a random move. If the move improves the situation, it is always accepted. Otherwise, the algorithm accepts the move with some probability less than 1.



The probability also decreases as the "temperature" T goes down: "bad moves are more likely to be allowed at the start when temperature is high, and they become more unlikely as T decreases

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  current ← problem.INITIAL
  for t = 1 to ∞ do
      T ← schedule(t)
      if T = 0 then return current
      next ← a randomly selected successor of current
      ΔE ← VALUE(current) – VALUE(next)
      if ΔE > 0 then current ← next
      else current ← next only with probability e^(−ΔE/T)
```

3. Local and Stochastic Beam Search

https://youtu.be/Ad29SjJ1GwA?si=9KtW0UNIGrgrQJcT

Keeping just one node in memory might seem to be an extreme reaction to the problem of memory limitations. The local beam search algorithm keeps track of k states rather than just one. It begins with k randomly generated states. At each step, all the successors of all k states are generated. If any one is a goal, the algorithm halts. Otherwise, it selects the k best successors from the complete list and repeats.

In its simplest form, local beam search can suffer from a lack of diversity among the k states-they can quickly become concentrated in a small region of the state space, making the search little more than an expensive version of hill climbing.

To address the diversity problem, a variant known as stochastic beam search is proposed. This variant draws inspiration from stochastic hill climbing.

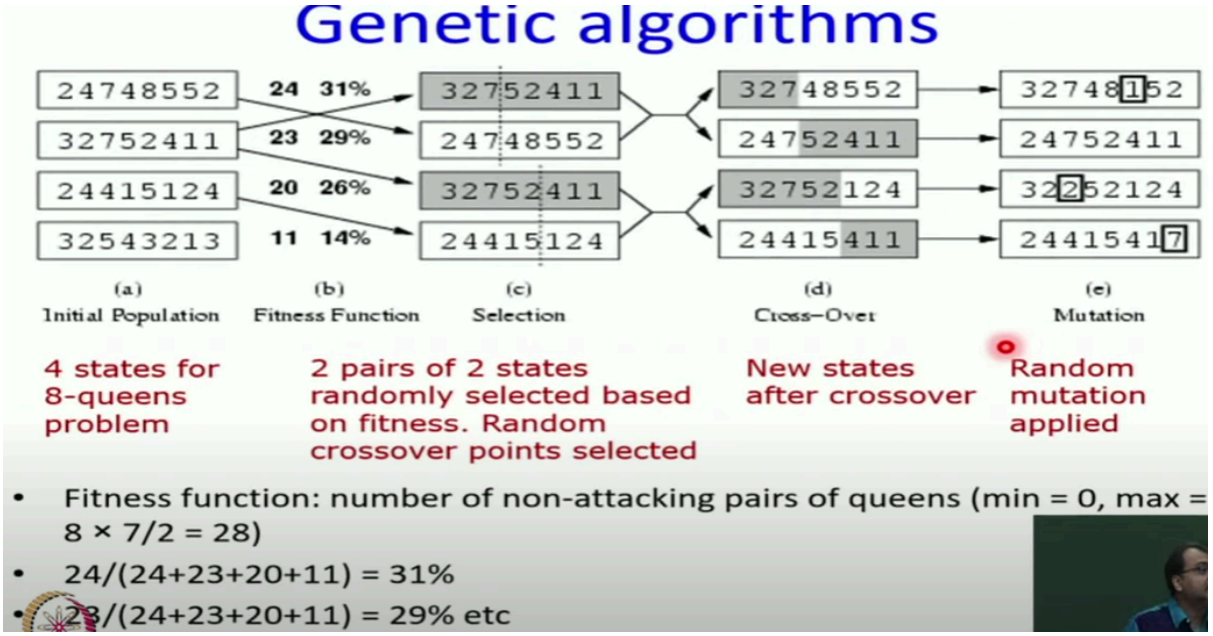
In stochastic beam search, instead of strictly selecting the k best successors from the pool of candidates, the algorithm randomly selects k successors. The probability of selecting a particular successor increases with its value or quality. This means that even solutions with lower quality have a chance to be chosen, promoting exploration and diversity in the search process.

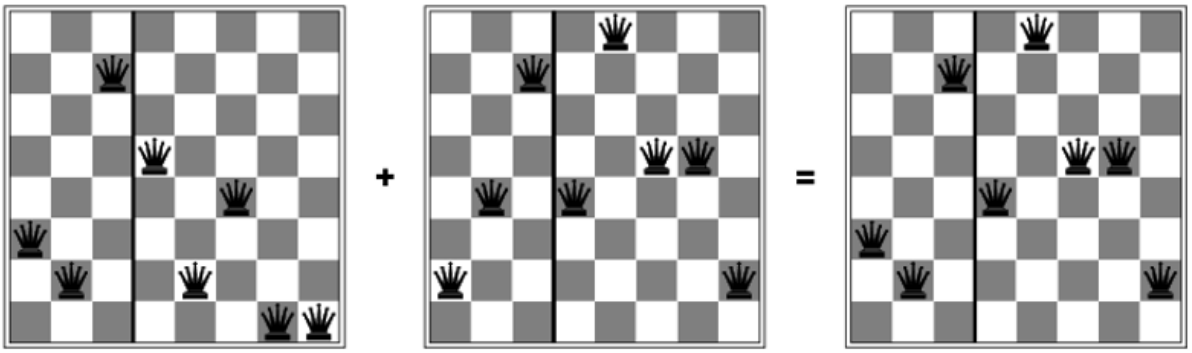
Stochastic beam search bears some resemblance to the process of natural selection, whereby the "successors" (offspring) of a "state" (organism) populate the next generation according to its "value" (fitness).

4. Genetic Algorithm

https://youtu.be/ZdVL5k_Jkbg?si=khN9U_kbEP-RmOII

A genetic algorithm (or GA) is a variant of stochastic beam search in which successor states are generated by combining two parent states, rather than by modifying a single state. The analogy to natural selection is the same as in stochastic beam search, except now we are dealing with sexual rather than asexual reproduction.





5. What is Game Theory ? Domain ?

https://youtu.be/FFzdXJ49KAI?si=inP4CPPvKPqqxjEK

In AI, .'games are usually of a rather specialised kind-what game theorists call deterministic, turn-taking, two-player, zero-sum games of perfect information. In our terminology, this means deterministic, fully observable environments in which there are two agents whose actions must alternate and in which the utility values at the end of the game are always equal and opposite.

For example, if one player wins a game of chess (+I), the other player necessarily loses (-1). It is this opposition between the agents' utility functions that makes the situation adversarial.

Game playing was one of the first tasks undertaken in AI. By 1950, almost as soon as computers became programmable, chess had been tackled by Konrad Zuse and by Alan Turing. Since then, there has been steady progress in the standard of play, to the point that machines have surpassed humans in checkers and Othello, have defeated human champions (although not every time) in chess and backgammon, and are competitive in many other games.

•Average branching factor is 35
•In an average game each player might make 50 moves
•In order to examine the complete game tree, we would have to examine 35^{100} positions
Therefore it is highly impossible to select a move during the lifetime of the opponent

Some kind of heuristic search procedure is necessary

•Improve the generate procedure so that only good moves (or paths) are generated
•Improve the test procedure so that the best moves (or paths) will be recognized and explored
•Minmax search procedure
It is depth-first, depth-limited search procedure

6. Optimal Decisions in Games

https://youtu.be/FFzdXJ49KAI?si=GNocLF5C5DSxyn0X

We will consider games with two players, whom we will call MAX and MIN. MAX moves first, and then they take turns moving until the game is over. At the end of the game, points are awarded to the winning player and penalties are given to the loser.

A game can be formally defined as a kind of search problem with the following components:

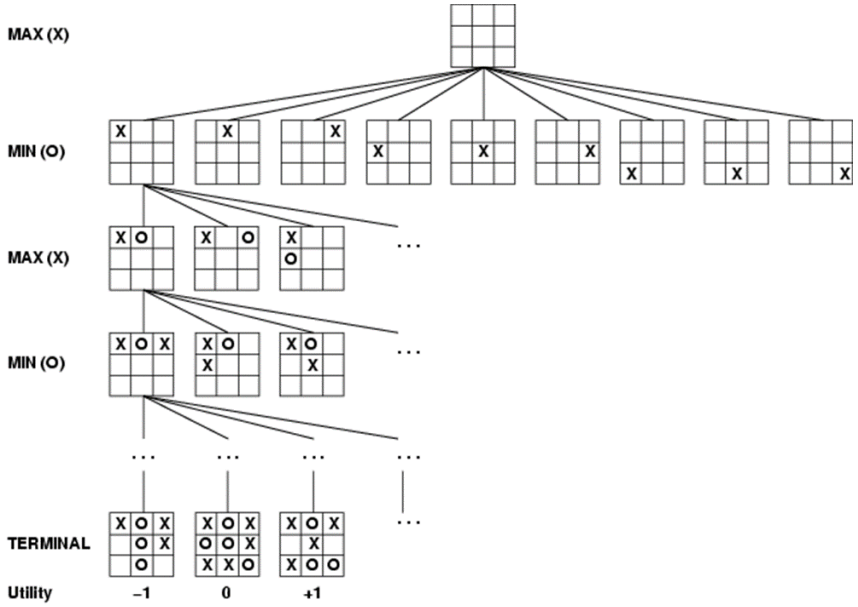
The **initial** state, which includes the board position and identifies the player to move.

A **successor** function, which returns a list of (move, state) pairs, each indicating a legal move and the resulting state.

A **terminal** test, which determines when the game is over. States where the game has ended are called terminal states.

A **utility** function (also called an objective function or payoff function), which gives a numeric value for the terminal states. In chess, the outcome is a win, loss, or draw, with values +1, -1, or 0. Some games have a wider ,variety of possible outcomes; the payoffs in backgammon range from +I92 to -192.

GAME TREE

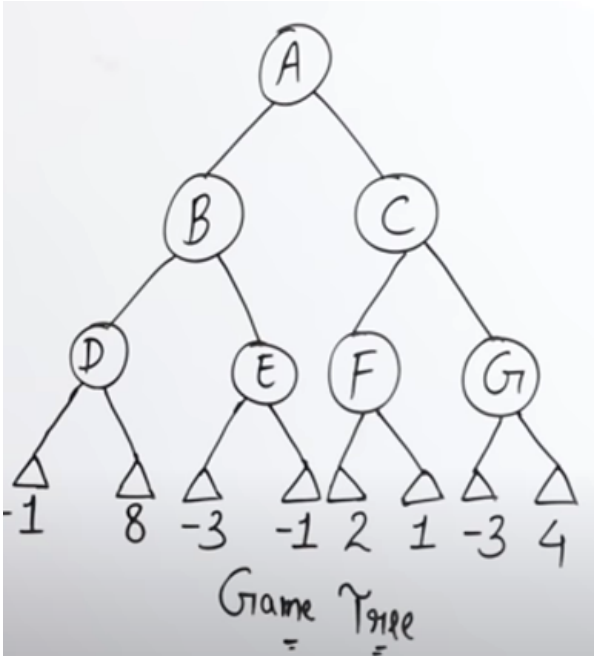


The initial state and the legal moves for each side define the game tree for the game. Figure 6.1 shows part of the game tree for tic-tac-toe (noughts and crosses). From the initial state, MAX has nine possible moves. Play alternates between MAX'S placing an x and MIN'S placing an o until we reach leaf nodes corresponding to ter.mina1 states such that one player has three in a row or all the squares are filled. The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are assumed to be good for MAX and bad for MIN (which is how the players get their names). It is MAX'S job to use the search tree (particularly the utility of terminal states) to determine the best move.

7. MIN MAX ALGORITHM

https://youtu.be/Ntu8nNBL28o?si=Bfczk3KNX4kERUty

The minimax algorithm (Figure 6.3) computes the minimax decision from the current state. It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations. The recursion proceecls all the way down to the leaves of the tree, and then the minimax values are backed up through the tree as the recursion Unwinds.



If the maximum depth of the tree is m, and there are b legal moves at each point, then the time complexity of the minimax algorithm is O(b m). The space complexity is O(bm) for an algorithm that generates all successors at once, or O(m) for an algorithm that generates successors one at a time (see page 76).

```
function MINIMAX-DECISION(state) returns an action
    v ← MAX-VALUE(state)
    return the action in SUCCESSORS(state) with value v

function MAX-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s))
    return v

function MIN-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← ∞
    for a, s in SUCCESSORS(state) do
        v ← MIN(v, MAX-VALUE(s))
    return v
```

- **Properties of Mini-Max algorithm:**
- **Complete**- Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.
- **Optimal**- Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity**- As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is **$O(b^m)$**, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity**- Space complexity of Mini-max algorithm is also similar to DFS which is **O(bm)**.

8. Alpha Beta pruning

https://youtu.be/dEs_kbvu_0s?si=iM89DB0lcyMx-abE

Problems With Minmax Algorithm
How does Alha Beta works
Example
Time Complexity

Unit 3

Unit 3

1. Why do we need knowledge based agents ?

Humans, it seems, know things and do reasoning. Knowledge and reasoning are also important for artificial agents because they enable successful behaviours that would be very hard to achieve otherwise. We have seen that knowledge of action outcomes enables problem solving agents to perform well in complex environments.

A reflex agent could only find its way from Arad to Bucharest by dumb luck. The knowledge of problem-solving agents is, however, very specific and inflexible. A chess program can calculate the legal moves of its king, but does not know in any useful sense that no piece can be on two different squares at the same time. Knowledge-based agents can benefit from knowledge expressed in very general forms, combining and recombining information to suit myriad purposes

EXAMPLE

Knowledge and reasoning also play a crucial role in dealing with partially obsemable environments. A knowledge-based agent can combine general knowledge with current percepts to infer hidden aspects of the current state prior to selecting actions.

For example, a physician diagnoses a patient-that is, infers a disease state that is not directly observable prior to choosing a treatment. Some of the knowledge that the physician uses is in the form of rules learned from textbooks and teachers, and some is in the form of patterns of association that the physician may not be able to consciously describe. If it's inside the physician's head, it counts as knowledge.

Understanding natural language also requires inferring hidden state, namely, the intention of the speaker. When we hear, "John saw the diamond through the window and coveted it," we know "it" refers to the diamond and not the window-we reason, perhaps unconsciously, with our knowledge of relative value. Similarly, when we hear, "John threw the brick through the window and broke it," we know "it" refers to the window

Flexibility :

1. They can able to accept new task
2. They can achieve competence quickly by being told or learning new knowledge,
3. They can adapt to changes in the environment by updating the relevant knowledge

2. What are Knowledge Based Agents?

The central component of a knowledge-based agent is its knowledge base, or **KB**. Informally, a knowledge base is a set of sentences. (Here "sentence" is used as a technical term. It is related but is not identical to the sentences of English and other natural languages.) Each sentence is expressed in a language called a knowledge representation language and represents LANGUAGE
some assertion about the world.

There must be a way to add new sentences to the knowledge base and a way to query what is known. The standard names for these tasks are TELL and ASK, respectively. Both tasks may involve inference-that is, deriving new sentences from old.
In logical agents, **inference** must obey the fundamental requirement that when one ASKS a question of the knowledge base, the answer should follow
from what has been told (or rather, TELL^^) to the knowledge base previously

```
function KB-AGENT(percept) returns an action
  static: KB, a knowledge base
          t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION- QUERY(^))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

Figure 7.1 A generic knowledge-based agent.

Figure 7.1 shows the outline of a knowledge-based agent program. Like all our agents, it takes a percept as input and returns an action. The agent maintains a knowledge base, KB, which may initially contain some background knowledge. Each time the agent program is called, it does three things.

First, it TELLS the knowledge base what it perceives.

Second, it ASKS the knowledge base what action it should perform. In the process of answering this query, extensive reasoning may be done about the current state of the world, about the outcomes of possible action sequences, and so on.

Third, the agent records its choice with TELL and executes the action. The second TELL is necessary to let the knowledge base know that the hypothetical action has actually been executed.

knowledge level, where we need specify only what the agent knows and what its goals are, in order to fix its behavior.

For example, an automated taxi might have the goal of delivering a passenger to Marin County and might know that it is in San Francisco and that the Golden Gate Bridge is the only link between the two locations. Then we can expect it to cross the Golden Gate Bridge because it knows that that will achieve its goal.

Notice that this analysis is independent of how the taxi works at the **implementation level**. It doesn't matter whether its geographical knowledge is implemented as linked lists or pixel maps, or whether it reasons by manipulating strings of symbols stored in registers or by propagating noisy signals in a network of neurons.

3. WUMPUS WORLD

https://youtu.be/SHfP8SPOEEw?si=bM3qKy3mm-P9iPxJ

4. Logic , Model and Entailment in Wumpus Model

https://youtu.be/zOCTxedhf_c?si=cC_4hpcHBLdtzi3v (9 min)

Syntax

Knowledge bases consist of sentences. These sentences are expressed according to the syntax of the representation language, which specifies all the sentences that are well formed. The notion of syntax is clear enough in ordinary arithmetic:
"x + y = 4" is a well-formed sentence, whereas "x2y+ =" is not. The syntax of logical languages (and of arithmetic, for that matter) is usually designed for writing papers and books.

There are literally dozens of different syntaxes, some with lots of Greek letters and exotic mathematical symbols, and some with rather visually appealing diagrams with arrows and bubbles.

Semantics

A logic must also define the semantics of the language. Loosely speaking, semantics has to do with the "meaning" of sentences. In logic:, the definition is more precise. The semantics of the language defines the truth of each sentence with respect to each possible world. For example, the usual semantics adopted for arithimetic specifies that the sentence "x + y = 4" is true in a world where x is 2 and y is 2, but false in a world where x is 1 and y is 1.' In standard logics, every sentence must be either true or false in each possible world-there is no "in bet~een."~

Model

When we need to be precise, we will use the term model in place of "possible world." (We will also use the phrase "m is a model of a!" to mean that sentence a is true in model m.)

Informally, we may think of x and y as the number of men and women sitting at a table playilng bridge, for example, and the sentence x + y = 4 is true when there are four in total;
formally, the possible models are just all possible assignments of numbers to the variables x: and y. Each such assignment fixes the truth of any sentence of arithmetic whose variables are x and y.
M(alpha) s a set of all model for the alpha

Entailment

the idea that a sentence follows logically from another sentence. In mathematical notation, we write as to mean that the sentence a! entails the sentence p.

$$\alpha \models \beta$$

The formal definition of entailment is this: a I= p if and only if, in every model in which a is true, ,p is also true. Another way to say this is that if a! is true, then p must also be true.

Informally, the truth of ,beta is "contained" in the truth of a. The relation of entailment is familiar from. arithmetic; we are happy with the idea that the sentence x + y = 4 entails the sentence 4 = x: + y. Obviously, in any model where x + y = 4--such as the model in which x is 2 am~d y is 2-it is the case that 4 = x t y.

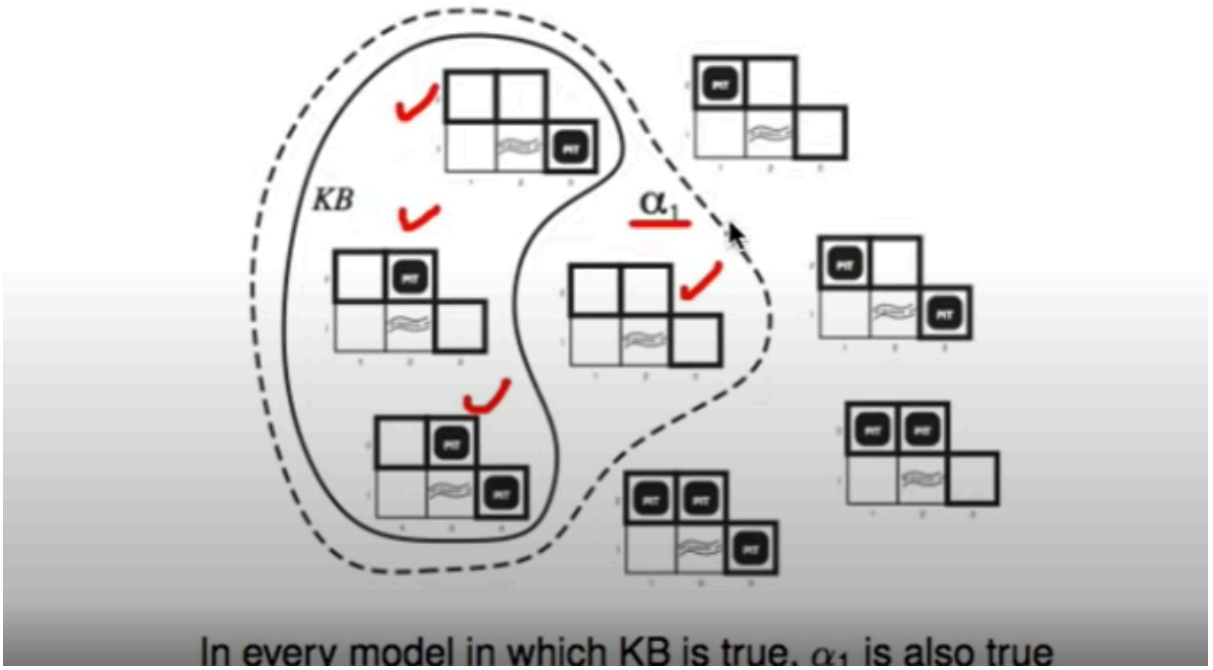
- $\alpha \models \beta$ iff in every model where α is true, β is also true.
- $\alpha \models \beta$ iff $M(\alpha) \subseteq M(\beta)$.
- examples:
  1. $(x = 0) \models (xy = 0)$
  2. $(p = True) \models (p \vee q)$
  3. $False \models True$
  4. $(p \wedge q) \models (p \vee q)$

We can apply the same kind of analysis to the wumpus-world reasoning example given in the preceding section. Consider the situation in Figure 7.3(b): the agent has detected nothing in [1,1] and a breeze in [2,1 I.

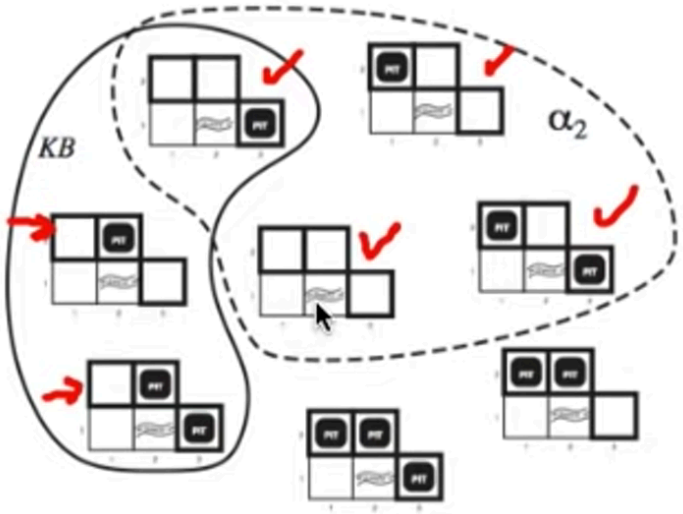
These precepts, combined with the agent's knowledge of the rules of the wumpus world (the PEAS description on page 197), constitute the KB.

the KB is false in any model in which [1,2] contains a pit, because there is no breeze in [1,1]. There are in fact just three models in which the KB is true, and these are shown as a subset of the models

Model the presence of pits in squares [1,2][2,2] and [3,1]. α_1 models no pits in [1,2]. Then, does $KB \models \alpha_1$?



Model the presence of pits in squares [1,2][2,2] and [3,1] with the restriction α_2 of no pits in [2,2]. Does $KB \models \alpha_2$?



In every model in which KB is true, α_2 is not necessarily true.

The preceding example not only illustrates entailment, but also shows how the definition of entailment can be applied to derive conclusions-that is, to carry out logical inference. The inference algorithm illustrated in Figure 7.5 is called model checking, because it enumerates all possible models to check that a is true in all models in which KB is true.

In understanding entailment and **inference**, it might help to think of the set of all consequences of KB as a haystack and of a as a needle. Entailment is like the needle being in the haystack; inference is like finding it. This distinction is embodied in some formal notation:

$KB \vdash_i \alpha$

if an inference algorithm i can derive a from KB, we write
which is pronounced "a is derived from KB by i" or "2 derives a from KB."

An inference algorithm that derives only entailed sentences is called sound or truth-preserving. Soundness is a highly desirable property

Soundness: *i* is sound if whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$

Completeness: *i* is complete if whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$

The property of completeness is also desirable: an inference algorithm is complete if it can derive any sentence that is entailed.

5. Propositional Logic

https://youtu.be/6490tKrGEic?si=UYvITvdeASE9x9Lj

SYNTAX

multiplication has higher precedence than addition. The order of precedence in propositional logic is (from highest to lowest): ¬, A, V, ⇒, and ⇔. I-Ience, the sentence

$$\neg P \vee Q \wedge R \Rightarrow S$$

is equivalent to the sentence

$$((\neg P) \vee (Q \wedge R)) \Rightarrow S .$$

Semantics

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

WUMPUS

Now that we have defined the semantics for propositional logic, we can construct a knowledge base for the wumpus world. For simplicity, we will deal only with pits; the wumpus itself is left as an exercise. We will provide enough knowledge to carry out the inference that was done informally in Section 7.3.

First, we need to choose our vocabulary of proposition symbols. For each i, j:

- Let $P_{i,j}$ be true if there is a pit in [i,j].
- Let $B_{i,j}$ be true if there is a breeze in [i,j].

The knowledge base includes the following sentences, each one labeled for convenience:

- There is no pit in [1,1]

$$R_1 : \quad \neg P_{1,1} .$$

- A square is breezy if and only if there is a pit in a neighboring square. This has to be stated for each square; for now, we include just the relevant squares:

$$R_2 : \quad B_{1,1} \quad \Leftrightarrow \quad (P_{1,2} \vee P_{2,1}) .$$
$$R_3 : \quad B_{2,1} \quad \Leftrightarrow \quad (P_{1,1} \vee P_{2,2} \vee P_{3,1}) .$$

- The preceding sentences are true in all wumpus worlds. Now we include the breeze percepts for the first two squares visited in the specific world the agent is in, leading up to the situation in Figure 7.3(b).

$$R_4 : \quad \neg B_{1,1} .$$
$$R_5 : \quad B_{2,1} .$$

Recall that the aim of logical inference is to decide whether $KB \models a$ for some sentence a. For example, is $P_{2,2}$ entailed? Our first algorithm for inference will be a direct implementation of the definition of entailment: enumerate the models, and check that a is true in every model in which KB is true. For propositional logic, models are assignments of true or false to every proposition symbol. Returning to our wumpus-world example, the relevant proposition symbols are $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, and $P_{3,1}$. With seven symbols, there are $2^7 = 128$ possible models; in three of these, KB is true (Figure 7.9). In those three models, $\neg P_{1,2}$ is true, hence there is no pit in [1,2]. On the other hand, $P_{2,2}$ is true in two of the three models and false in one, so we cannot yet tell whether there is a pit in [2,2].

6. Logical Equivalence

The first concept is logical equivalence: two sentences α and β are logically equivalen if they are true in the same set of models. We write this as $\alpha \equiv \beta$. For example, we can easily show (using truth tables) that $\mathbf{P} \wedge \mathbf{Q}$ and $\mathbf{Q} \wedge \mathbf{P}$ are logically equivalent; other equivalences are shown in Figure 7.11. They play much the same role in logic as arithmetic identities do in ordinary mathematics. An alternative definition of equivalence is as follows: for any two sentences α and β,

$$\alpha \equiv \beta \quad \text{if and only if} \quad \alpha \models \beta \text{ and } \beta \models \alpha .$$

(Recall that $\models$ means entailment.)

- Two sentences are logically equivalent iff true in same models: α ≡ ß iff α ⊨ β and β ⊨ α

$$\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{de Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{de Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}$$

VALIDTY

The second concept we will need is validity. A sentence is valid if it is true in *all* models. For example, the sentence $\mathbf{P} \vee \neg P$ is valid. Valid sentences are also known as tautologies—they are *necessarily* true and hence vacuous. Because the sentence Due is true in all models, every valid sentence is logically equivalent to Due.

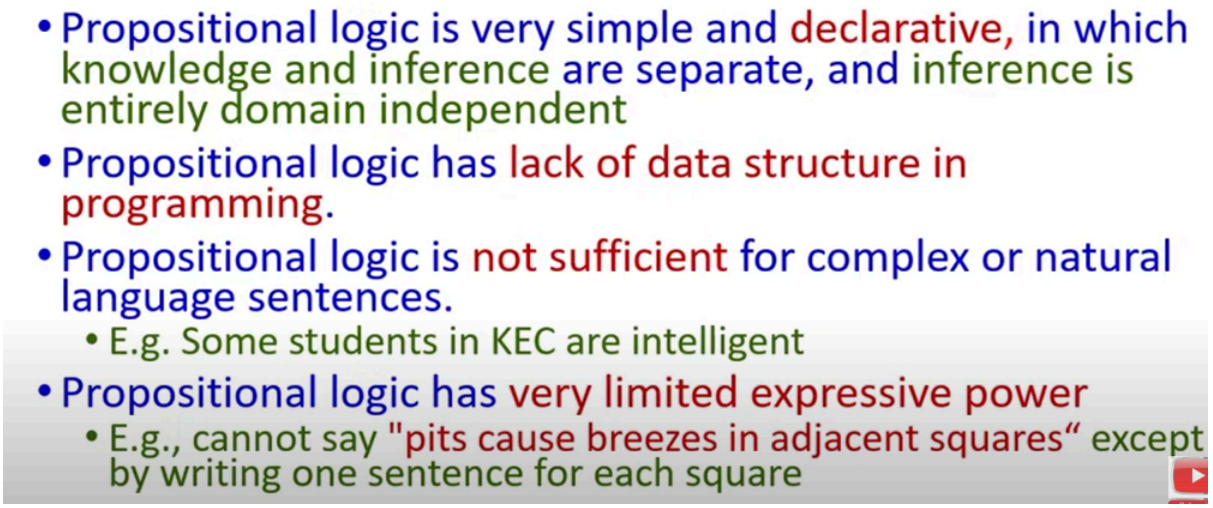
Satisfiable

The final concept we will need is **satisfiability.** A sentence is satisfiable if it is true in *some* model. For example, the knowledge base given earlier, $(R_1 \wedge R_2 \wedge R_3 \wedge$ [redacted] $\wedge$ [redacted] is satisfiable because there are three models in which it is true, as shown in Figure 7.9. If a sentence α is true in a model m, then we say that m **satisfies** α, or that m **is a model of** α. Satisfiability can be checked by enumerating the possible models until one is found that satisfies the sentence. Determining the satisfiability of sentences in propositional logic was

https://youtu.be/aQYxyYfCJ4Q?si=cCQD0KCly-piLYfb

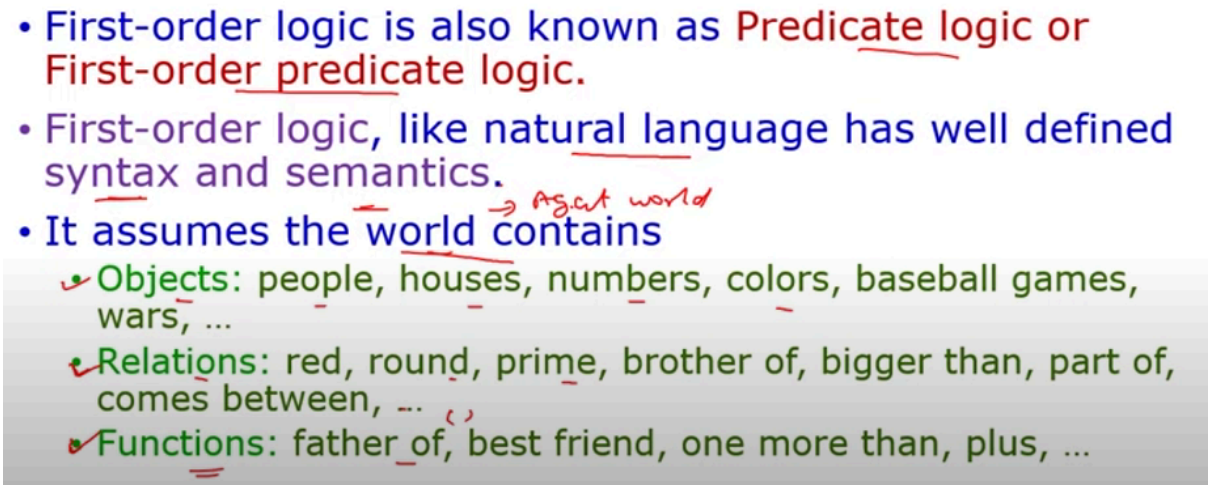
- Decide whether each of the following sentences is valid, un-satisfiable or neither.
  - Smoke $\Rightarrow$ Smoke
  - Smoke $\Rightarrow$ Fire
  - (Smoke $\Rightarrow$ Fire)($\neg$ Smoke$\Rightarrow$ $\neg$ Fire)
  - Smoke $\vee$ Fire $\vee$ $\neg$ Fire
  - ((Smoke$\wedge$ Heat) $\Rightarrow$Fire) $\Leftrightarrow$ ((Smoke $\Rightarrow$ Fire) $\vee$ (Heat $\Rightarrow$ Fire))
  - (Smoke $\Rightarrow$ Fire) $\Rightarrow$(Smoke$\wedge$ Heat) $\Rightarrow$Fire)
  - Big $\vee$ Dumb $\vee$ (Big $\Rightarrow$ Dumb)
  - (Big $\vee$ Dumb) $\vee$ $\neg$ Dumb

7. Drawback of propositional logic



8. What is FOL

https://youtu.be/wD6-My8mL-U?si=w6mCIO91LiSMgjlk



SYNTAX

CPFV CEQ

- Constants KingJohn, 2, NUM,...
- Predicates Brother, >,...
- Functions Sqrt, LeftLegOf,...
- Variables x, y, a, b,...
- Connectives $\neg, \Rightarrow, \wedge, \vee, \Leftrightarrow$
- Equality =
- Quantifiers $\forall, \exists$

TWO TYPES OF SENTENCES
https://youtu.be/VV0U5j_hUB0?si=UelBs2vCDqxhKnKb

TYPES OF QUANTIFIERS
https://youtu.be/3WYCk2Pm1G4?si=QVWdk7tyJQnOCSWz

First-order logic includes one more way to make atomic sentences, other than using a predicate and terms as described earlier. We can use the **equality symbol** to make statements to the effect that two terms refer to the same object. For example,

$$Father(John) = Henry$$

says that the object referred to by $Father(John)$ and the object referred to by $Henry$ are the same. Because an interpretation fixes the referent of any term, determining the truth of an

PROPERTIES OF QUANTIFIERS

- $\forall x\ \forall y$ is the same as $\forall y\ \forall x$
-
- $\exists x\ \exists y$ is the same as $\exists y\ \exists x$
-
- $\exists x\ \forall y$ is not the same as $\forall y\ \exists x$
-
- $\exists x\ \forall y$ Loves(x,y)
  - "There is a person who loves everyone in the world"
- $\forall y\ \exists x$ Loves(x,y)
  - "Everyone in the world is loved by at least one person"

- Quantifier duality: each can be expressed using the other
- $\forall x$ Likes(x,IceCream) $\quad\quad \neg\exists x\ \neg$Likes(x,IceCream)
- $\exists x$ Likes(x,Broccoli) $\quad\quad \neg\forall x\ \neg$Likes(x,Broccoli)

Examples

https://youtu.be/x5GfV8ORetQ?si=DRrflEPWJSnKrpP1

Unit 4

Unit 4

Pg 166

1. What is Constraint Satisfaction Problem and its variety's?

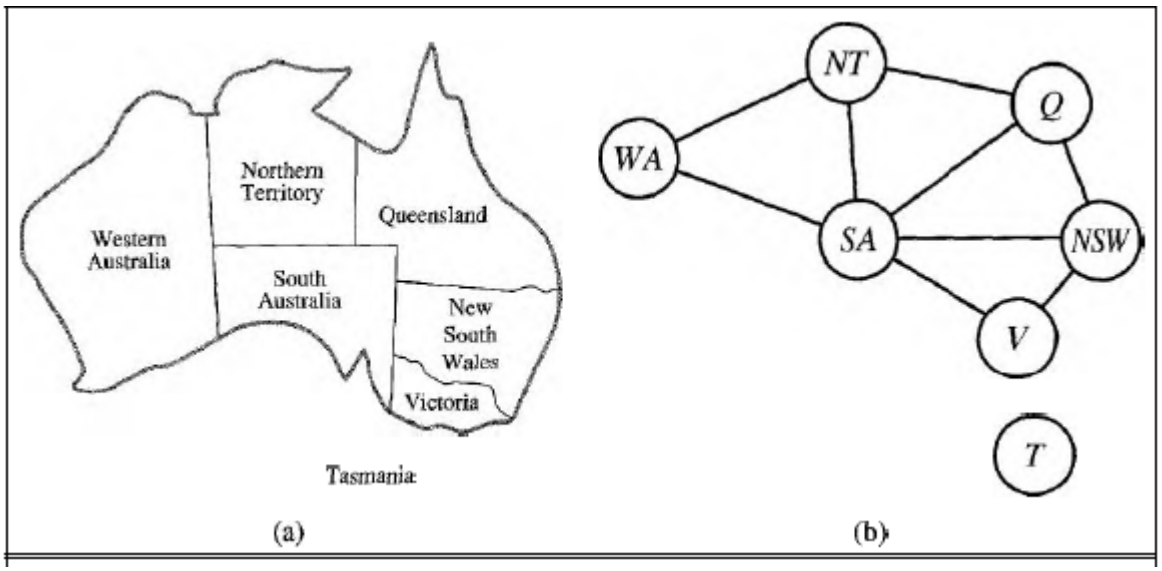
https://youtu.be/AgyCSmDVk5s?si=IszmDJ7lrC5OmKjM

https://youtu.be/yYQIG9iVvVQ?si=oSAEziv4ZndhVGBK

Formally speaking, a constraint satisfaction problem (or CSP) is defined by a set of variables, XI, X2,. . . , Xn, and a set of constraints, C1, (72,. . . , C,. Each variable Xi has a nonempty domain Di of possible values. Each constraint Ci involves some subset of the variables and specifies the allowable combinations of values for that subset. A state of the problem is defined by an assignment of values to some or all of the variables, {Xi = vi, Xj = vj, . . .). An assignment that does not violate any constraints is called a consistent or legal assignment. A complete assignment is one in which every variable is mentioned, and a solution to a CSP is a complete assignment that satisfies all the constraints. Some CSPs also require a solution that maximizes an objective function.

Suppose that, having tired of Romania, we are looking at a map of Australia showing each of its states and territories, as in Figure 5.l(a), and that we are given the task of coloring each region either red, green, or blue in such a way that no neighboring regions have the same color. To formulate this as a CSP, we define the variables to be the regions: WA, NT, Q, NSW, V, SA, and T. The domain of each variable is the set
{red, green, blue).

The constraints require neighboring regions to have distinct colors; for example, the allowable combinations for WA and NT are the pairs
{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)) .
(The constraint can also be represented more succinctly as the inequality WA # NT, provided the constraint satisfaction algorithm has some way to evaluate such expressions.) There are many possible solutions, such as
{ WA= red, NT = green, Q = red, NSW = green, V= red, SA= blue, T= red).



Treating a problem as a CSP confers several important benefits. Because the representation of states in a CSP conforms to a standard pattern-that is, a set of variables with assigned values-the successor function and goal test can be written in a generic way that applies to all CSPs. Furthermore, we can develop effective, generic heuristics that require no additional, domain-specific expertise. Finally, the structure of the constraint graph can be used to simplify the solution process, in some cases giving an exponential reduction in complexity. The CSP representation is the first, and simplest, in a series of representation schemes that will be developed throughout the book.

2. What is Cryptarithmetic

   https://youtu.be/g_T19glKzyU?si=SDJgs9UykYsfuuEf

3. Incremental CSF?

It is fairly easy to see that a CSP can be given an incremental formulation as a standard search problem as follows:
0 Initial state: the empty assignment {), in which all variables are unassigned.
0 Successor function: a value can be assigned to any unassigned variable, provided that it does not conflict with previously assigned variables.
0 Goal test: the current assignment is complete.
0 Path cost: a constant cost (e.g., 1) for every step.

Every soluf on must be a complete assignment and therefore appears at depth n if there are

n variables. Furthermore, the search tree extends only to depth n. For these reasons, depthfirst search algorithms are popular for CSPs. (See Section 5.2.) It is also the case that the path by which a solution is reached is irrelevant. Hence, we can also use a complete-state formulation, in which every state is a complete assignment that might or might not satisfy the constraints. Local search methods work well for this formulation.

Finite domain

The simplest kind of CSP involves variables that are discrete and have finite domains. Map-coloring problems are of this kind. The 8-queens problem described in Chapter 3 can also be viewed as a finite-domain CSP, where the variables Q1, . . . , Q8 are the positions of each queen in columns 1, . . . ,8 and each variable has the domain {1,2,3,4,5,6,7,8). If the maximum domain size of any variable in a CSP is d, then the number of possible complete assignments is O(dn)-that is, exponential in the number of variable

Infinite domain

Discrete variables can also have infinite domains-for example, the set of integers or the set of strings. For example, when scheduling construction jobs onto a calendar, each job's start date is a variable and the possible values are integer numbers of days from the current date. With infinite domains, it is no longer possible to1 describe constraints by enumerating all allowed combinations of values. Instead, a constraint language must be used. For example, if Jobl, which takes five days, must precede Jobs, then we would need a constraint language of algebraic inequalities such as StartJobl + 5 5 StartJobs

Continuous domain

Constraint satisfaction problems with continuous (domains are very common in the real world and ar~ew idely studied in the field of operations research. For example, the scheduling

of experiments on the Hubble Space Telescope requires very precise timing of observations; the start and finish of each observation and maneuver are continuous-valued variables that must obey a variety of astronomical, precedence, and power constraints.

4. Backtracking Search CSF?

5. Variable and Value Ordering?

   https://youtu.be/xYRDEw59eIM?si=j-33Bzc8hR5a7fKN

MRV
Degree Heuristic
Least Constraining value

6. What is Forward Checking and arc Consistency?

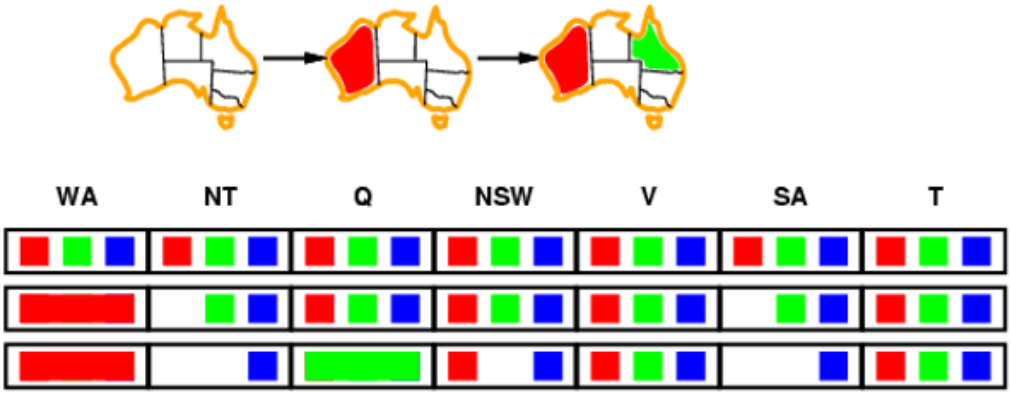
   https://youtu.be/GSNrDCnhe7w?si=mHQzlw3r7EDCMjPv

   https://youtu.be/hPeh9-Zf1J8?si=AjTf_To79mulYLqw

So far our search algorithm considers the constraints on a variable only at the time that the variable is chosen by SELECT-UNASSIGNED-VARIABLE. But by looking at some of the constraints earlier in the search, or even before the search has started, we can drastically reduce the search space.

Forward checking

One way to make better use ofx constraints during search is called forward checking. Whenever a variable X is assigned, the forward checking process looks at each unassigned variable Y that is connected to X by a constraint and deletes from Y's domain any value that is inconsistent with the value chosen for X. Figure 5.6 shows the progress of a map-coloring search with forward checking. There are two important points to notice about this example. First, notice that after assigning WA = red and Q = green, the domains of NT and SA are reduced to a single value; we have eliminated branching on these variables altogether by propagating information from WA and Q. The MRV heuristic, which is an obvious partner for forward checking, would automatically select SA and NT next. (Indeed, we can view forward checking as an efficient way to incrementally compute the information that the MRV heuristic needs to do its job.) A second point to notice is that, after V = blue, the domain of SA is empty. Hence, forward checking has detected that the partial assignment { WA = red, Q = green, V = blue) is inconsistent with the constraints of the problem, and the algorithm will therefore backtrack immediately.
MRV heuristic needs to do its job.) A second point to notice is that, after V = blue, the domain of SA is empty. Hence, forward checking has detected that the partial assignment { WA = red, Q = green, V = blue) is inconsistent with the constraints of the problem, and the algorithm will therefore backtrack immediately.



Constraint Propagation

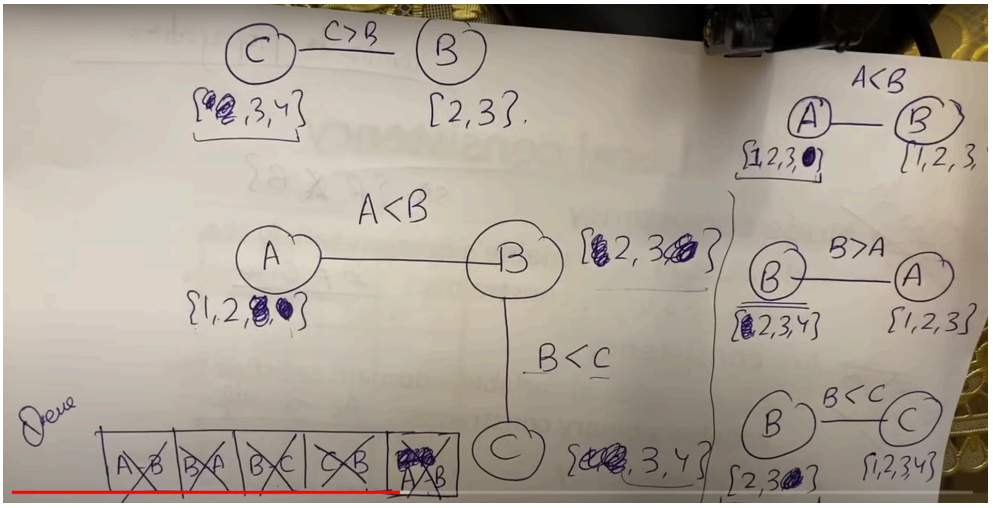
Although forward checking detects many inconsistencies, it does not detect all of them. For example, consider the third row of Figure 5.6. It shows that when WA is red and Q is green, both NT and SA are forced to be blue. But they are ,adjacent and so cannot have the same value. Forward checking does not detect this as an inconsistency, because it does not look far enough ahead. Constraint propagation is the general term for propagating the implications of a constraint on one variable onto other variables; xn this case we need to propagate from WA and Q onto NT and SA, (as was done by forward checking) and then onto the constraint between NT and SA to detect the inconsistency

ARC CONSISTENCY

The idea of arc consistency provides a fast method of constraint propagation that is substantially stronger than forward checking. Here, "arc" refers to a directed arc in the constraint graph, such as the arc from SA to NS W. Given the current domains of SA and NS W, the arc is consistent if, for every value x of SA, there is some value y of NS W that is consistent with x. In the third row of Figure 5.6, the current domains of SA and NSW are {blue) and {red, blue) respectively. For SA = blue, there is a consistent assignment for NSW, namely, NSW = red; therefore, the arc from SA to I1JS'W is consistent. On the other hand, the reverse arc from NS W to SA is not consistent: for Ithe assignment NS W = blue, there is no consistent assignment for SA. The arc can be made consistent by deleting the value blue from the domain of NSW.

We can also apply arc consistency to the arc from SA to NT at the same stage in the search process. The third row of the table in Figure 5.6 shows that both variables have the domain {blue). The result is that blue must be deleted from the domain of SA, leaving the domain empty. Thus, applying arc consistency has resulted in early detection of an inconsis-tency that is not detected by pure forward checking

Arc consistency checking can be applied either as a preprocessing step before the beginning of the search process, or as a propagation step (like forward checking) after every assignment during search. (The latter algorithm is sometimes called MAC, for Maintaining Arc Consistency.) In either case, the process must be applied repeatedly until no more inconsistencies remain. This is because, whenever a value is deleted from some variable's domain to remove an arc inconsistency, a new arc inconsistency could arise in arcs pointing to that variable. The full algorithm for arc consistency, AC-3, uses a queue to keep track of the arcs that need to be checked for inconsistency.



7. What is Local Search For CSP?

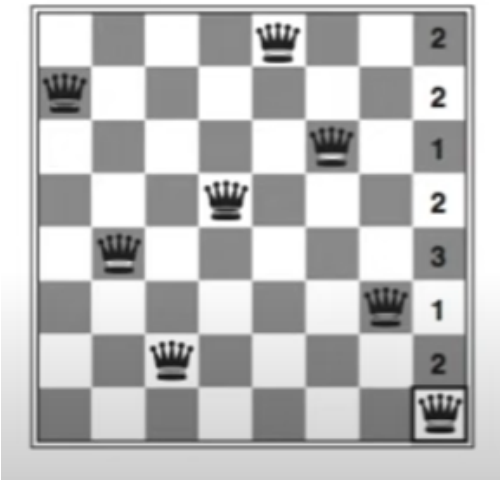
https://youtu.be/lCrHYT_EhDs?si=LDRfpVDT9StgssS9
25 min

Local-search algorithms (see Section 4.3) turn out to be very effective in solving many CSPs.They use a complete-state formulation: the initial state assigns a value to every variable, and the successor function usually works by changing the value of one variable at a time.
For example, in the 8-queens problem, the initial state might be a random configuration of 8 queens in 8 columns, and the successor function picks one queen and considers moving it elsewhere in its column. Another possibility would be start with the 8 queens, one per column in a permutation of the 8 rows, and to generate a successor by having two queens swap rows.

In choosing a new value for a variable, the most obvious heuristic is to select the value MIN-CONFLICTS that results in the minimum number of conflicts with other variables-the min-conflicts heuristic. The algorithm is shown in Figure 5.8 and its

application to an 8-queens problem is diagrammed in Figure 5.9 and quantified in Figure 5.5.
Min-conflicts is surprisingly effective for many CSPs, particularly when given a reasonable initial state. Its performance is shown in the last column of Figure 5.5. Amazingly, on the n-queens problem, if you don't count the initial placement of queens, the runtime of minconflicts is roughly independent of problem size. It solves even the million-queens problem in an average of 50 steps (after the initial assignment). This remarkable observation was the stimulus leading to a great deal of research in the 1990s on local search and the distinction between easy and hard problems, which we take up in Chapter 7. Roughly speaking, n-queens is easy for local search because solutions are densely distributed throughout the state space.

Min-conflicts also works well for hard problems. For example, it has been used to schedule observations for the Hubble Space Telescope, reducing the time taken to schedule a week of observations from three weeks (!) to around 10 minute

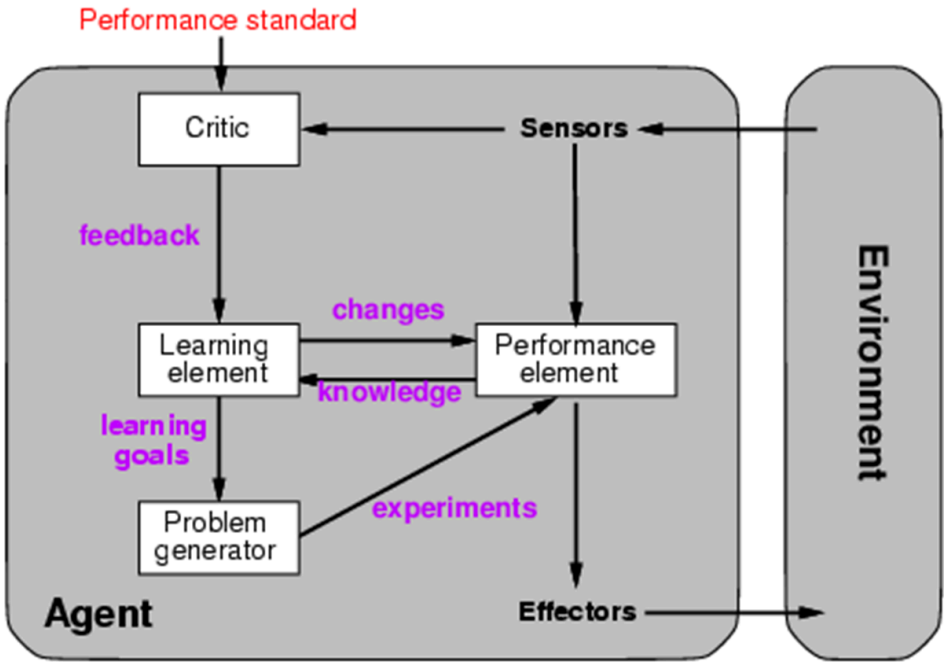


Unit 5

Unit 5

1. What is Learning and Learning agent?

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E



A learning agent can be divided into four conceptual components, as shown in Figure 2.15. The most important distinction is between the learning element, which is re-

sponsible for making improvements, and the performance element, which is responsible for selecting external actions.

Performance and Learning element

The performance element is what we have previously considered to be the entire agent: it takes in percepts and decides on actions. The learning element uses feedback from the critic on how the agent is doing and determines how the performance element should be modified to do better in the future.

The design of the learning element depends very much on the design of the performance element. When trying to design an agent that learns a certain capability, the first question is not "How am I going to get it to learn this?" but "What kind of performance element will my agent need to do this once it has learned how?'Given an agent design, learning mechanisms can be constructed to improve every part of the agent.

Critic

The critic tells the learning element how well the agent is doing with respect to a fixed performance standard. The critic is necessary because the percepts themselves provide no indication of the agent's success. For example, a chess program could receive a percept indicating that it has checkmated its opponent, but it needs a performance standard to know that this is a good thing; the percept itself does not say so. It is important that the performance standard be fixed.

Problem generator

The last component of the learning agent is the problem generator. It is responsible for suggesting actions that will lead to new and informative experiences. The point is that if the performance element had its way, it would keep doing the actions that are best, given what it knows. But if the agent is willing to explore a little, and do some perhaps suboptimal actions in the short run, it might discover much better actions for the long run. The problem generator's job is to suggest these exploratory actions

Example

To make the overall design more concrete, let us return to the automated taxi example.

The performance element consists of whatever collection of knowledge and procedures the taxi has for selecting its driving actions. The taxi goes out on the road

and drives, using this performance element. The critic observes the world and passes information along to the learning element. For example, after the taxi makes a quick left turn across three lanes of traffic, the critic observes the shocking language used by other drivers. From this experience, the learning element is able to formulate a rule saying this was a bad action, and the performance element is modified by installing the new rule. The problem generator miglht identify certain areas of behavior in need of improvement and suggest experiments, such as trying out the brakes on different road surfaces under different conditions.

2. What is learning Element and types of feedback?

We know that a learning agent can be thought of as containing a performance element that decides what actions to take and a learning element that modifies the performance element so that it makes better decisions. Machine learning researchers have come up with a large variety of learning elements.

To understand them, it will help to see how their design is affected by the context in which they will operate.

The design of a learning element is affected by three major issues:

1. Which components of the performance element are to be learned:

    - In many cases, not all components of the performance element need to be learned. Instead, the focus might be on learning only certain parameters or policies that directly impact the agent's decision-making process.
    - For example, in a reinforcement learning setting, the learning element might focus on learning the values of state-action pairs (Q-values) or the policy that governs the agent's behavior.

2. What feedback is available to learn these components:

    - Feedback refers to the information provided to the learning element about the quality of its decisions.
    - The type and availability of feedback greatly influence the learning process. Feedback can be explicit, such as rewards in reinforcement learning, or implicit, such as observing the success or failure of actions.

- For example, in supervised learning, the learning element receives explicit feedback in the form of labeled training data, while in unsupervised learning, feedback might be less direct and more implicit.

3. What representation is used for the components:

    - Representation refers to how the components of the performance element, such as states, actions, and feedback, are encoded or represented in the learning process.
    - Different learning algorithms may require different types of representations. For instance, in deep learning, neural networks are used to represent complex relationships in data, while in symbolic learning, logic-based representations are used to capture symbolic relationships.

Types of Feedback

Several components that make up the performance element of an agent:

1. Direct mapping from conditions on the current state to actions.
2. Means to infer relevant properties of the world from the percept sequence.
3. Information about the world's evolution and the results of possible actions.
4. Utility information indicating the desirability of world states.
5. Action-value information indicating the desirability of actions.
6. Goals that describe classes of states whose achievement maximizes the agent's utility.

The type of feedback available for learning is usually the most important factor in determining the nature of the learning problem that the agent faces. The field of machine learning usually distinguishes three cases: supervised, unsupervised, and reinforcement learning.

Supervised Learning

The problem of supervised learning involves learning a function from examples of its inputs and outputs. Cases (I), (2), and (3) are all instances of supervised learning problems.

In (I), the agent learns condition-action rule for braking-this is a function from states to a Boolean output (to brake or not to brake), In (2), the agent learns a function from images to a Boolean output (whether the image contains a bus). In (3), the theory of braking is a function from states and braking actions to, say, stopping distance in feet.

Notice that in cases (1) and (2), a teacher provided the correct output value of the examples; in the third, the output value was available directly from the agent's percepts. For fully observable environments, it will always be the case that an agent can observe the effects of its actions and hence can use supervised learning methods to learn to predict them. For partially observable environments, the problem is more difficult, because the immediate effects might be invisible.

Unsupervised Learning

The problem of unsupervised learning involves learning patterns in the input when no specific output values are supplied. For example, a taxi agent might gradually develop a concept of "good traffic days" and "bad traffic days" without ever being given labelled examples of each.

Reinforcement Learning

Rather than being told what to do by a teacher, a reinforcement learning agent must learn from reinforcement.' For example, the lack of a tip at the end of the journey (or a hefty bill for rear-ending the car in front) gives the agent some indication that its behavior is undesirable. Reinforcement learning typically includes the subproblem of learning how the environment works

3. What is Inductive Learning?

Pg. 678

An algorithm for deterministic supervised learning is given as input the correct value of the unknown function for particular inputs and must try to recover the unknown function or some-thing close to it. More formally, we say that an example is a pair (x, f (z)), where x is the input and f(x) is the output of the function applied to x. The task of pure inductive inference (or induction) is this:

Given a collection of examples of f , return a function h that approximates f .
The function h is called a hypothesis.
A good hypothesis will generalize well-that is, will predict unseen examples correctly.
This is the fundamental problem of induction.

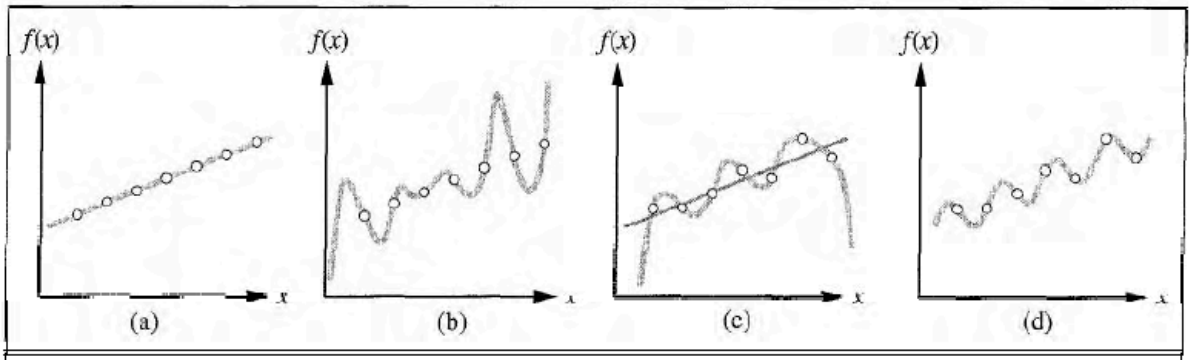


Figure 18.1 (a) Example $(x, f(x))$ pairs and a consistent, linear hypothesis. (b) A consistent, degree-7 polynomial hypothesis for the same data set. (c) A different data set that admits an exact degree-6 polynomial fit or an approximate linear fit. (d) A simple, exact sinusoidal fit to the same data set.

Figure: 18.1 shows a familiar example: fitting a function of a single variable to some data points. The examples are (x, f (x)) pairs, where both x and f (x) are real numbers. We choose the hypothesis space H-the set of hypotheses we will consider-to be the set of polynomials of degree at most k, such as 3x2 $ 2, x17 - 4x3, and so on.

Figure 18.l(a) shows some data with an exact fit by a straight line (a polynomial of degree l). The line is called a consistent hypothesis because it agrees with all the data. Figure 18.l(b) shows a high-degree polynomial that is also consistent with the same data. This illustrates the first issue in inductive learning: how do we choose from among multiple consistent hypotheses?
One answer is **Ockham's2** razor: prefer the simplest hypothesis consistent with the data. Intuitively, this makes sense, because hypotheses that are no simpler than the data themselves are failing to extract any pattern from the data. Defining simplicity is not easy, but it seems reasonable to say that a degree- 1 polynomial is simpler than a degree- 12 polynomial

Figure 18.l(c) shows a second data set. There is no consistent straight line for this data set; in fact, it requires a degree-6 polynomial (with 7 parameters) for an exact fit. There are just 7 data points, so the polynomial has as many parameters as there are data points: thus, it does not seem to be finding any pattern in the data and we do not expect it to generalize well. It might be better to fit a simple straight line that is not exactly consistent but might make reasonable predictions.We have to settle for a trade off.

One should keep in mind that the possibility or impossibility of finding a simple, consistent hypothesis depends strongly on the hypothesis space chosen. Figure 18.l(d) shows that the data in (c) can be fit exactly by a simple function of the form ax + b + c sin x. This example shows the importance of the choice of hypothesis space. A hypothesis space consisting of polynomials of finite degree cannot represent

sinusoidal functions accurately, so a learner using that hypothesis space will not be able to learn from sinusoidal data.

There is a tradeoff between the expressiveness of a hypothesis space and the complexity of finding simple, consistent hypotheses within that space. For example, fitting straight lines to data is very easy; fitting high-degree polynomials is harder; and fitting Turing machines is very hard indeed because determining whether a given

Turing machine is consistent with the data is not even decidable in general. A second reason to prefer simple hypothesis spaces is that the resulting hypotheses may be simpler to use-that is, it is faster to compute h(x) when h is a linear function than when it is an arbitrary Turing maclhine program

4. What is Supervised Learning and its types?

Supervised learning is a type of machine learning where the algorithm learns from labeled data, consisting of input-output pairs, to make predictions or decisions on unseen data. In supervised learning, the algorithm is provided with a training dataset containing examples with known input-output mappings, and the goal is to learn a mapping from inputs to outputs that can generalize to new, unseen data.

There are two main types of supervised learning:

Classification:

a. In classification tasks, the goal is to predict a categorical label or class label for each input example.
b. The output variable or target variable is discrete and represents different categories or classes.
c. Examples of classification tasks include spam email detection, sentiment analysis, image classification, and medical diagnosis.
d. Common algorithms for classification include logistic regression, decision trees, random forests, support vector machines (SVM), k-nearest neighbors (KNN), and neural networks.

Regression:

e. In regression tasks, the goal is to predict a continuous numerical value or quantity for each input example.
f. The output variable or target variable is continuous and represents a real-valued quantity.

g. Examples of regression tasks include predicting house prices, stock prices, temperature forecasting, and demand forecasting.
h. Common algorithms for regression include linear regression, polynomial regression, decision trees, random forests, support vector regression (SVR), and neural networks.

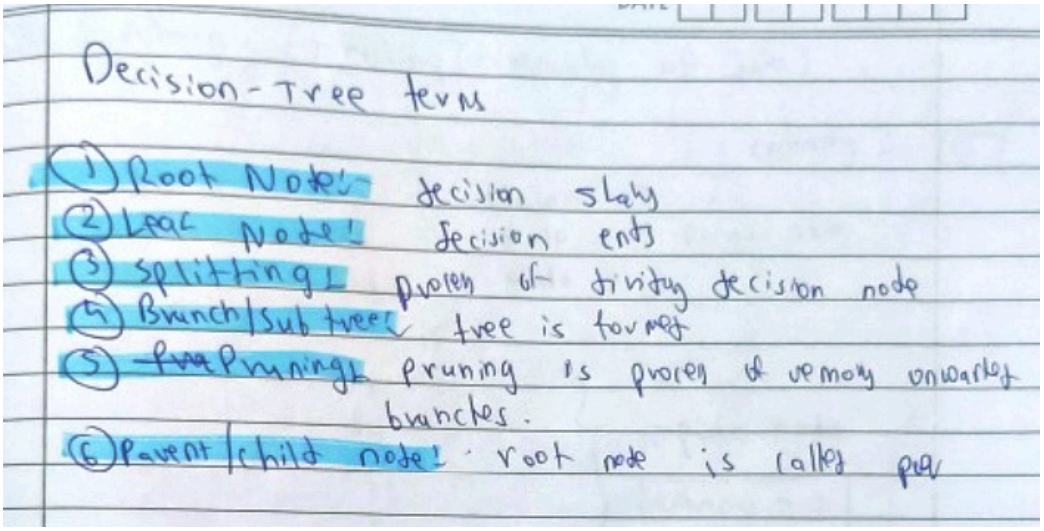
5. What is Decision Tree?

https://youtu.be/CWzpomtLqqs?si=UqtcCsWknGZM3qvi

A decision tree takes as input an object or situation described by a set of attributes and returns a "decision the predicted output value for the input. The input attributes can be discrete or continuous. For now, we assume discrete inputs. The output value can also be discrete or continuous; learning a discrete-valued function is called classification learning; learning a continuous function is called regression.

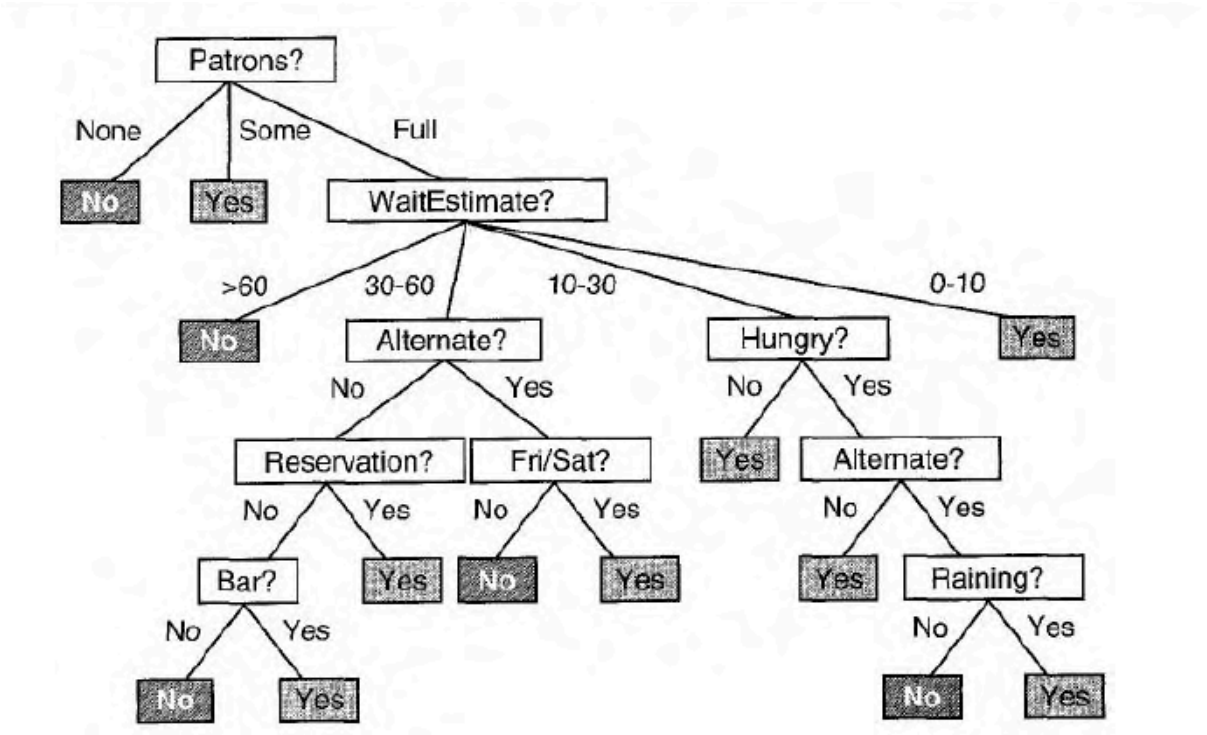
A decision tree reaches its decision by performing a sequence of tests. Each internal node in the tree corresponds to a test of the value of oine of the properties, and the branches from the node are labeled with the possible values of the test. Each leaf node in the tree specifies the value to be returned if that leaf is reached

A somewhat simpler example is provided by the problem of whether to wait for a table at a restaurant. The aim here is to learn a definition for the **goal predicate** Will Wait. In setting this up as a learning problem, we first have to state what attributes are available to describe examples in the domain.



1. Alternate: whether there is a suitable alternative restaurant nearby.
2. *Bar*: whether the restaurant has a comfortable bar area to wait in.
3. *Fri/Sat*: true on Fridays and Saturdays.
4. *Hungry*: whether we are hungry.
5. Patrons: how many people are in the restaurant (values are None, Some, and Full).
6. Price: the restaurant's price range ($, $$, $$$).
7. Raining: whether it is raining outside.
8. Reservation: whether we made a reservation.
9. Type: the kind of restaurant (French, Italian, Thai, or burger).
10. *WaitEstimate*: the wait estimated by the host (0–10 minutes, 10–30, 30–60, >60).

Examples are processed by the tree starting at the root and following the appropriate branch until a leaf is reached. For instance, an example with Patrons = Full and WaitEstimate = 0-10 will be classified as positive (i.e., yes, we will wait for a table).



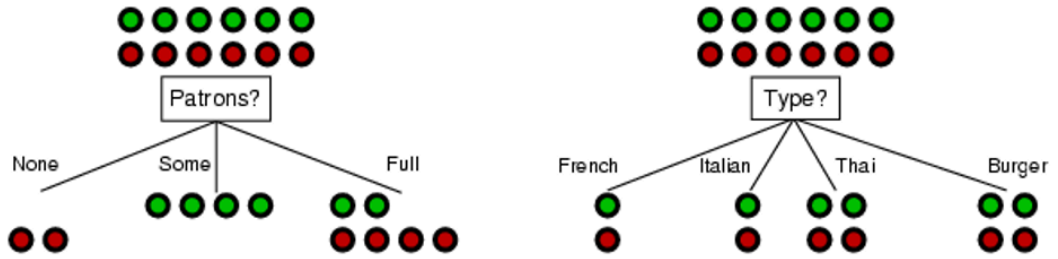
igure 18.2 A decision tree for deciding whether to wait for a table

Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*Wait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

An example for a Boolean decision tree consists of a vector of' input attributes, X, antd a single Boolean output value 3. A set of examples (XI,y l) . . . , (XI2y, 12) is shown in Figure 18.3
The positive examples are the ones in which the goal Will Wait is true (XI, X3, . . .); the negative examples are the ones in which it is false (X2, X5, . . .). The complete set of examples is called the training set.

The basic idea behind the DECISION-TREE is to test the most important attribute first. By "most important," we mean the one that makes the most difference to the classification of an example. That way, we hope to get to the correct classification with a small number of tests, meaning that all paths in the tree will be short and the tree as a whole will be small.

Figure 18.4 shows how the algorithm gets started. We are given 12 training examples, which we classify into positive and negative sets. We then decide which attribute to use as the first test in the tree. Figure 18.4(a) shows that Type is a poor attribute, because it leaves us with four possible outcomes, each of which has the same number of positive and negative examples.



On the other hand, in Figure 18.4(b) we see that Patrons is a fairly important attribute, because if the value is None or Some, then we are left with example sets for which we can answer definitively (No and Yes, respectively).. If the value is Full, we are left with

a mixed set of examples. In general, after the first attribute test splits up the examples, each outcome is a new decision tree learning problem in itself, with fewer examples and one fewer attribute.

There are four cases to consider for these recursive problems:

1. If there are some positive and some negative examples, then choose the best attribute to split them. Figure 18.4(b) shows Hungry being used to split the remaining examples.
2. If all the remaining examples are positive (or all. negative), then we are done: we can answer Yes or No. Figure 18.4(b) shows examples of this in the None and Some cases.
3. If there are no examples left, it means that no such example has been observed, and we return a default value calculated from the majority classification at the node's parent.
4. If there are no attributes left, but both positive and neg,ative examples, we have a problem.

It means that these examples have exactly the same description, but different classifications. This happens when some of the data are incorrect; we say there is noise in the data. It also happens either when the attributes do not give enough information to describe the situation fully, or when the domain is truly nondeterministic. One simple way out of the problem is to use a majority vote.

Choosing the attribute test / Information Gain

The scheme used in decision tree learning for selecting attributes is designed to minimize the depth of the final tree. The idea is to pick the attribute that goes as far as possible toward providing an exact classification of the examples. A perfect attribute divides the examples into sets that are all positive or all negative

1. Entropy:
    - Entropy is a measure of impurity or disorder in a dataset.
    - In the context of decision trees, entropy is used to quantify the uncertainty in the target variable (class labels) at a particular node.
    - Mathematically, the entropy of a dataset with respect to a target variable Y is calculated using the following formula:
    - Entropy is maximum when all classes are equally distributed, and it decreases as the dataset becomes more homogeneous (i.e., contains mostly examples of one class).
2. Information Gain:
    - Information gain is a measure of the effectiveness of a particular attribute in classifying the data.

- It quantifies the reduction in entropy achieved by splitting the dataset on a particular attribute.
- The attribute with the highest information gain is chosen as the splitting criterion at each node of the decision tree.
- Higher information gain indicates that splitting the data on the attribute X results in greater homogeneity within the resulting subsets.

Entropy measures impurity in the data and information gain measures reduction in impurity in the data. The feature which has minimum impurity will be considered as the root node.

Information gain is used to decide which feature to split on at each step in building the tree. The creation of sub-nodes increases the homogeneity, that is decreases the entropy of these nodes. The more the child node is homogeneous, the more the variance will be decreased after each split. Thus Information Gain is the variance reduction and can calculate by how much the variance decreases after each split.

6. What is Unsupervised Learning that challenges and some applications?

Unsupervised learning is a type of machine learning where the algorithm is tasked with finding patterns or structure in a dataset without explicit guidance or labeled examples. In unsupervised learning, the algorithm explores the data on its own to discover inherent relationships or groupings within the data. Unlike supervised learning, where the algorithm is provided with labeled examples and learns to predict labels for new data points, unsupervised learning operates on unlabeled data and focuses on finding hidden structure or patterns.

One common technique in unsupervised learning is clustering. Clustering is a process of grouping similar data points together based on their features or characteristics. The goal of clustering is to partition the dataset into clusters or groups, where data points within the same cluster are more similar to each other than to those in other clusters. Clustering algorithms aim to maximize intra-cluster similarity while minimizing inter-cluster similarity.

Challenges

Desired Output Not Available:

a. Unlike supervised learning where the algorithm is provided with labeled examples, unsupervised learning operates on unlabeled data. This lack of desired output makes it more challenging for the algorithm to learn meaningful patterns or structures in the data.

b. Additionally, deciding on a stopping condition for the clustering process can be difficult as there is no clear objective or target to optimize towards.

Need for Human Intervention:

c. Unsupervised learning algorithms may require human intervention to interpret and understand the patterns or clusters discovered in the data.
d. Human expertise and domain knowledge are often necessary to make sense of the clusters and correlate them with real-world phenomena. This can add to the cost and complexity of the learning process.

Cluster Validation:

e. Validating the quality and usefulness of the clusters obtained from unsupervised learning can be challenging.
f. Unlike in supervised learning where the accuracy of predictions can be measured using labeled data, there is no ground truth or output measure to confirm the effectiveness of clustering results.
g. Evaluating the validity of clusters often involves subjective judgment and domain-specific knowledge, which can be difficult to quantify objectively.

Scalability:

h. Unsupervised learning algorithms may face scalability issues when dealing with large datasets. Some clustering algorithms, for example, may become computationally expensive as the size of the dataset increases.
i. Scaling unsupervised learning algorithms to handle big data requires efficient implementations, distributed computing frameworks, or approximation techniques.

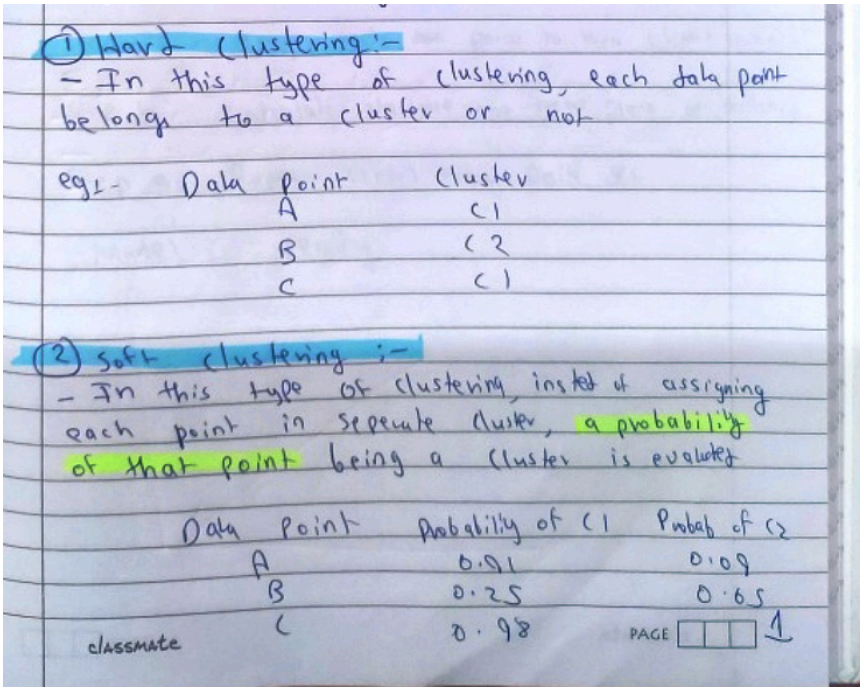
Robustness to Noise and Outliers:

j. Unsupervised learning algorithms may be sensitive to noise and outliers in the data, which can adversely affect the quality of clustering or pattern discovery.
k. Noisy or outlier-prone data points can distort cluster boundaries and lead to incorrect or unreliable results. Robust techniques are needed to mitigate the impact of noise and outliers on unsupervised learning algorithms.

Types of Clutering

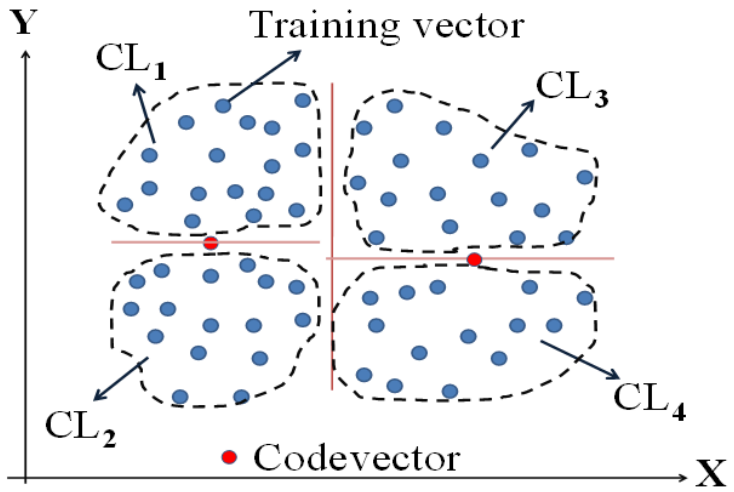
1. K-means clustering: A centroid-based clustering algorithm where data points are grouped into K clusters based on their distances to the centroids of these clusters.

2. Hierarchical clustering: A method that builds a hierarchy of clusters by recursively merging or splitting clusters based on their similarity.
3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise): A density-based clustering algorithm that identifies clusters as dense regions separated by areas of lower density.
4. Gaussian Mixture Models (GMM): A probabilistic clustering algorithm that models clusters as Gaussian distributions and assigns data points to clusters based on their probability densities.



7. What is K Means Clustering?

https://youtu.be/5FpsGnkbEpM?si=8_LmkW9vCpUlajBq



K-means clustering is one of the most popular and widely used unsupervised learning algorithms for clustering. It is a centroid-based clustering algorithm that partitions a dataset into K clusters, where each data point belongs to the cluster with the nearest mean (centroid). The goal of K-means clustering is to minimize the within-cluster variance, also known as inertia or sum of squared distances, which measures the compactness of clusters.

Here's how the K-means clustering algorithm works:

1. Initialization:
    - Choose the number of clusters, K, and randomly initialize K cluster centroids. These centroids can be randomly selected from the data points or generated using other initialization methods.
2. Assignment Step:
    - Assign each data point to the nearest cluster centroid based on the Euclidean distance between the data point and the centroids. Each data point is assigned to the cluster with the closest centroid.
3. Update Step:
    - Update the cluster centroids by calculating the mean of all data points assigned to each cluster. The new centroid position is the average of the data points in the cluster.
4. Convergence:
    - Repeat the assignment and update steps iteratively until convergence criteria are met. Convergence is typically reached when the cluster assignments and centroids no longer change significantly between iterations, or when a maximum number of iterations is reached.
5. Finalization:
    - Once convergence is achieved, the algorithm outputs the final cluster centroids and the cluster assignments for each data point.

Example : Youtube

Unit 6

Unit 6

1. What is an Expert System?

https://youtu.be/o0GFC6c_k4g?si=udAet-VjQtflGBXH

Example

https://youtu.be/bZ0R8bTCeaQ?si=2r6VHgdg-iWDrG9_

Advantages and Disadvantages

https://youtu.be/UD7ko385xgg?si=Or8e-z9KWTIN-ee2

2. What are the components of expert system and their roles of individual?

3. What are Expert System Shell , Characteristic and Featuers?

Expert system shells, also known as development environments or toolkits, provide a framework and set of tools for building, testing, and deploying expert systems. Here's how the key components you mentioned are typically integrated into expert system shells:

1. User Interface (UI):
    - Expert system shells often include user interface components that allow developers to design interactive interfaces for users to interact with the system.
    - These interfaces can range from simple command-line interfaces to more sophisticated graphical user interfaces (GUIs) or web-based interfaces.
    - The UI components may include forms, menus, buttons, text fields, and other elements for inputting queries, displaying results, and providing feedback to users.

2. Knowledge Base Format:
    - Expert system shells provide a structured format for representing and organizing declarative knowledge in the knowledge base.
    - The knowledge base format allows developers to input, edit, and manage the knowledge base efficiently, often using a specialized syntax or graphical editor within the shell.

3. Inference Engine:
    - The inference engine is the core component of the expert system shell responsible for reasoning and decision-making based on the knowledge stored in the knowledge base.
    - The inference engine may incorporate various reasoning techniques, such as forward chaining, backward chaining, or constraint satisfaction, depending on the capabilities and design of the expert system shell.

1. Knowledge Engineer:
    - The knowledge engineer is responsible for acquiring, structuring, and encoding domain-specific knowledge into the expert system.
    - They utilize the expert system shell to build a knowledge base tailored to a particular problem domain.
    - The knowledge engineer collaborates closely with domain experts to ensure the accuracy, relevance, and completeness of the knowledge base.

2. System Engineer:
    - The system engineer is responsible for designing and implementing the technical aspects of the expert system, including the user interface, knowledge base format, and inference engine.
    - They build the user interface components that allow users to interact with the system, design the structure and syntax of the knowledge base, and implement the algorithms and logic of the inference engine.

Characteristic

•Operates as an interactive system :
This means
–responds to questions
–asks for clarifications
–makes recommendations
–aids the decision-making process.

•Tools have ability to store and select (filter) knowledge
–storage and retrieval of knowledge
–mechanisms to expand and update knowledge base on a continuing basis

•Capability to make logical inferences based on knowledge stored
–simple reasoning mechanisms is used
–KB with means of exploiting the knowledge stored, else it is not useful

•Ability to explain reasoning
–remembers logical chain of reasoning therefore user may ask
–for explanation of a recommendation
–factors considered in recommendation
–enhances user confidence in recommendation and acceptance of expert system

•Systems are very domain-specific
–a particular system caters a narrow area of specialization; a medical expert system cannot be used to find faults in an electrical circuit
–quality of advice offered by an expert system is dependent on the amount of knowledge stored.

•Cost-effective alternative to human expert
–Expert systems have become increasingly popular because of their specialization, although in a narrow field.
–The small size of the domain makes encoding and storing the domain specific knowledge an economic process.
–specialists in many areas are scarce, and the cost of consulting them is high, an expert system catering to any of those areas can be considered to be a useful and cost-effective alternative, in the long run.

4. What is Goal Driven and Data Driven Reasoning?

Goal Driven

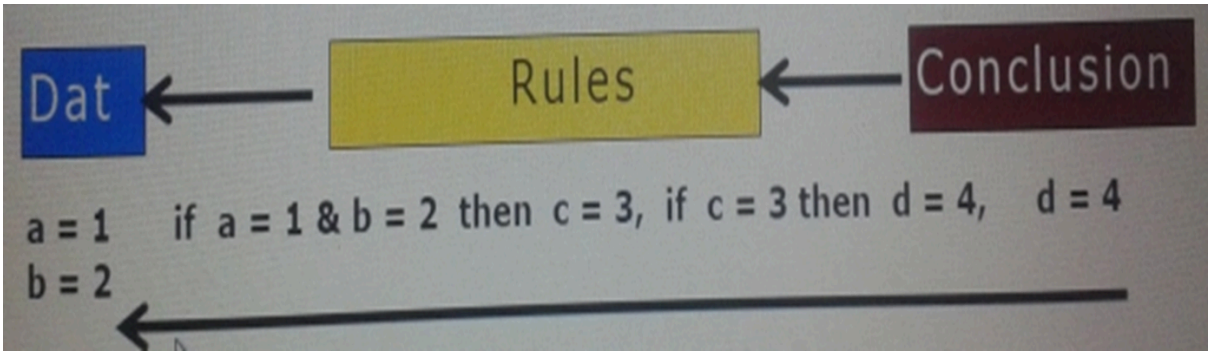
Goal-driven reasoning, also known as backward chaining, is a common and effective approach used in expert systems and other problem-solving domains. Let's delve deeper into how backward chaining works:

1. Starting with the Goal:
    - In backward chaining, the reasoning process starts with a given goal or desired outcome that the system aims to achieve.

2. Working Backwards:
    - The backward chaining algorithm works by iteratively working backward from the goal, attempting to find evidence or assertions that support the goal.
    - It begins by attempting to prove the goal by finding evidence in the knowledge base that directly supports or implies the goal. If such evidence is found, the goal is considered satisfied, and the process terminates.
    - If direct evidence for the goal is not available, the algorithm looks for other assertions or sub-goals that, if proven, could lead to the satisfaction of the original goal.

3. Adding New Assertions:
    - As the algorithm progresses, it adds new assertions or hypotheses to the set of goals to be proven based on the available knowledge and inference rules.
    - These new assertions may be derived from existing knowledge through logical inference, rule application, or other reasoning mechanisms.

4. Termination:
    - The backward chaining process terminates when one of the following conditions is met:
        - The original goal is proven or satisfied based on available evidence and assertions.
        - No further evidence or assertions can be found to support the current goals, indicating that the original goal cannot be satisfied given the available knowledge and assumptions.
5. Example:
    - For example, in a medical diagnosis system, the goal might be to determine the underlying cause of a patient's symptoms. The backward chaining algorithm would start with the hypothesis of possible diseases and work backward to find evidence in the patient's symptoms, medical history, and diagnostic tests that support or refute each disease hypothesis until a definitive diagnosis is reached.



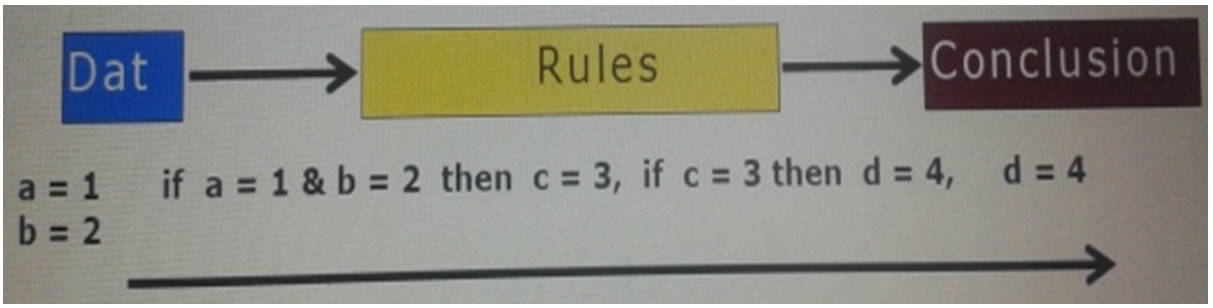
Data Driven

The data-driven approach, also known as forward chaining, offers an alternative method of reasoning in expert systems. Let's explore how forward chaining differs from backward chaining and how it accomplishes problem-solving:

1. Starting with Current State:
    - In contrast to backward chaining, forward chaining begins with the current state or initial set of assertions rather than a desired goal.
    - The system initiates with the available data or facts and seeks to derive new conclusions or assertions based on this starting point.

2. Progressing Toward Goal:
    - The forward chaining algorithm progresses incrementally by applying rules to the current set of assertions, seeking to move the system closer to a desired goal or solution.
    - By applying these rules and deriving new conclusions, the system updates its state and iteratively builds toward the desired goal.
3. Adding New Assertions:
    - As the algorithm progresses, it adds new assertions or conclusions to the set of known facts based on the rules that have been applied.
    - These new assertions serve to expand the system's understanding of the problem domain and move it closer to achieving the desired goal.
4. Termination:
    - The forward chaining process continues iteratively until one of the following conditions is met:
        - A predefined goal or termination condition is reached, indicating that the desired outcome has been achieved.
        - No further rules can be applied to derive new assertions, indicating that the system has reached a state where no additional progress can be made toward the goal.
5. Example:
    - For instance, in a diagnostic expert system, forward chaining might start with a set of patient symptoms and apply diagnostic rules to infer

possible diseases or conditions. It continues to apply rules and derive new conclusions until a definitive diagnosis is reached or no further inferences can be made.



5. What is Knowledge Acquisition?

https://youtu.be/BvpmYnF0NFA?si=rRB97-EAsRix8ReP

6. What are the techniques of knowledge Acquisition?

PHP MLS D

1. Protocol-generation Techniques:
    - These techniques involve conducting interviews with domain experts to gather information. Interviews can be unstructured (open-ended), semi-structured (combination of open and closed questions), or structured (predetermined questions). Additionally, reporting and observational techniques may be used to supplement interview data.

2. Protocol Analysis Techniques:
    - After interviews, protocol analysis techniques are applied to transcripts or text-based information to identify key knowledge objects such as goals, decisions, relationships, and attributes. This helps in understanding the structure and content of the knowledge obtained.

3. Hierarchy-generation Techniques (Laddering):
    - Laddering techniques involve creating, reviewing, and modifying hierarchical knowledge structures such as goal trees and decision networks. Different forms of ladders, such as concept ladder, attribute ladder, and composition ladder, may be used to represent hierarchical relationships.

4. Matrix-based Techniques:
    - Matrix-based techniques involve constructing grids or tables to represent relationships between problems and possible solutions. Frames may be used to represent the properties of concepts, facilitating the organization and analysis of knowledge.

5. Sorting Techniques:
    - Sorting techniques capture how people compare and order concepts, revealing knowledge about classes, properties, and priorities. This helps in understanding how experts perceive and categorize information.

6. Limited-information and Constrained-processing Tasks:
    - These techniques restrict the time and/or information available to experts when performing tasks. For example, the twenty questions technique prioritises key information in a domain by asking targeted questions.

7. Diagram-based Techniques:
    - Diagram-based techniques involve the generation and use of visual representations such as concept maps, state transition networks, event diagrams, and process maps. These diagrams help in capturing the "what, how, when, who, and why" of tasks and events, providing a holistic view of the domain.

7. Methods of Knowledge Representation?

•The knowledge acquired from the human expert must be encoded in such a way that it remains a faithful representation of what the expert knows, and it can be manipulated by a computer.

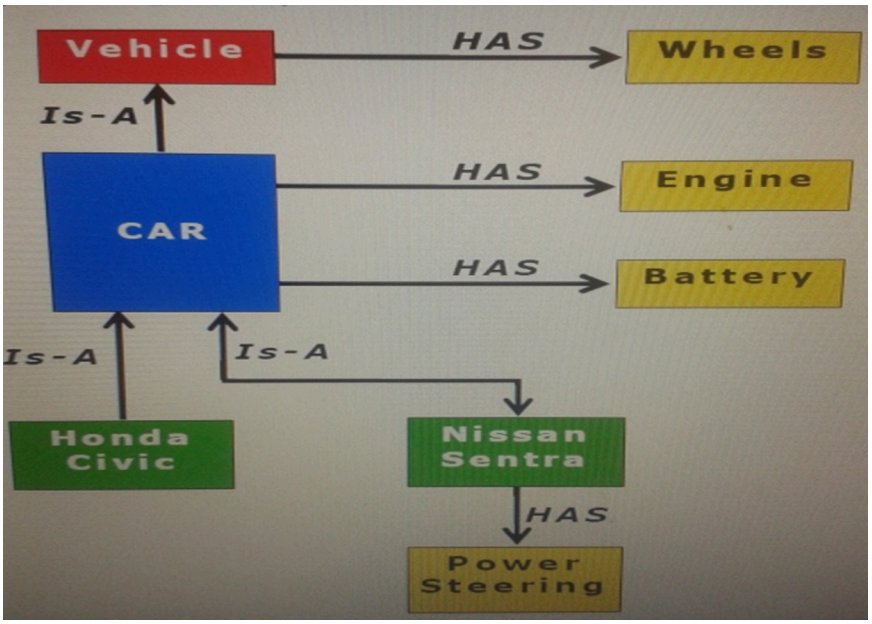
•Three common methods of knowledge representation evolved over the years are IF-THEN rules, semantic networks and frames.

If-Then rules

- Human experts usually tend to think along
- condition ⇒ action or Situation ⇒ conclusion
- Rules "if-then" are predominant form of encoding knowledge in expert systems. These are of the form :

  If *a1, a2, ... , an*

  Then *b1, b2, ... , bn*
- where each ***"ai" is a condition or situation, and each "bi" is an action or*** a conclusion.



Frame	Car
Inheritance Slot	Is-A
Value	Vehicle
Attribute Slot	Engine
Value	Vehicle
Value	1
Value	
Attribute Slot	Cylinders
Value	4
Value	6
Value	8
Attribute Slot	Doors
Value	2
Value	5
Value	4

Frame	Car
Inheritance Slot	Is-A
Value	Car
Attribute Slot	Make
Value	Honda
Value	
Value	
Attribute Slot	Year
Value	1989
Value	
Value	
Attribute Slot	
Value	
Value	
Value	